Automata, Computability, and Complexity Theory

Muchang Bahng

Summer 2025

Contents

1 Formal Languages **7**
1.1 Alphabets, Strings, and Languages . . . 7
Definition 1.1 (Alphabet) . . . 7
Definition 1.2 (Kleene Star) . . . 7
Definition 1.3 (Concatenation) . . . 7
Definition 1.4 (Formal Language) . . . 7
Definition 1.5 (Product of Formal Languages) . . . 7
1.2 Decision Problems as Languages . . . 7
1.3 Encodings . . . 7
1.4 Formal Grammars . . . 8
Definition 1.6 (Formal Grammar) . . . 8
Definition 1.7 (Derivation) . . . 8
Definition 1.8 (Language Derived From Grammar) . . . 8
Example 1.1 (A Grammar for Additive Expressions) . . . 8
Definition 1.9 (Parse Tree) . . . 9
Definition 1.10 (Transitive Closure) . . . 9
Definition 1.11 (Language Generated by Grammar) . . . 9
Definition 1.12 (Unrestricted) . . . 9

2 Finite Boolean Computation **11**
Definition 2.1 (AND Function) . . . 11
Definition 2.2 (OR Function) . . . 11
Definition 2.3 (NOT Function) . . . 11
Example 2.1 (Majority Function) . . . 11
Definition 2.4 (Universal Operation Set) . . . 12
Theorem 2.1 (AON Functions are Universal) . . . 12
2.1 Circuits . . . 12
Definition 2.5 (Boolean Circuit) . . . 12
Definition 2.6 (NOT Gate) . . . 13
Definition 2.7 (AND Gate) . . . 13
Definition 2.8 (OR Gate) . . . 13
Definition 2.9 (Computability of Circuits) . . . 14
Definition 2.10 (Universal Gate Set) . . . 14
Definition 2.11 (AON-CIRC) . . . 14
Theorem 2.2 (AON-CIRC is Universal) . . . 14
Example 2.2 (All Equals Function) . . . 15
2.2 Straight Line Programs . . . 15
Definition 2.12 (Straight Line Program) . . . 15
Definition 2.13 (Computability of Straight-Line Programs) . . . 16
Definition 2.14 (Universal Operator Set) . . . 16
Definition 2.15 (AON Straight Line Program) . . . 16
Theorem 2.3 (AON-SLP) . . . 16
Example 2.3 (XOR Function) . . . 16
2.3 NAND Circuits and Other Gate Sets . . . 17
Definition 2.16 (NAND Gate) . . . 17
Theorem 2.4 (NAND is Universal) . . . 17
Theorem 2.5 (Equivalence Between Circuits and Straight Line Programs) . . . 18
Definition 2.17 (NOR Gate) . . . 18
Definition 2.18 (XOR Gate) . . . 18

Example 2.4 (XOR from NAND) . . . 19
2.4 Conditionals . . . 19
Example 2.5 (Syntactic Sugar to Define Majority Function) . . . 19
Definition 2.19 (NAND-CIRC-PROC) . . . 20
Definition 2.20 (Multiplexor Gate) . . . 20
Definition 2.21 (Demultiplexor Gate) . . . 20
Definition 2.22 (Conditional Statement) . . . 20
Definition 2.23 (NAND-CIRC-IF) . . . 21
Theorem 2.6 (Removing Syntactic Sugar from NAND-CIRC-IF) . . . 21
Theorem 2.7 (Constant Plus Multiplexor is Universal) . . . 21
2.5 Arithmetic . . . 21
Theorem 2.8 (Addition using NAND-CIRC programs) . . . 22
Theorem 2.9 (Muliplication using NAND-CIRC programs) . . . 22
2.6 Code as Data, Data as Code . . . 22
Definition 2.24 (The Class $SIZE_{n,m}(s)$) . . . 22
2.7 Representing Programs as Strings . . . 23
Theorem 2.10 (Representing programs as strings) . . . 23
2.8 Counting Programs . . . 23
Theorem 2.11 (Counting programs) . . . 23
Theorem 2.12 (Counting argument lower bound) . . . 24
2.9 Tuples Representation . . . 24
Definition 2.25 (List of tuples representation) . . . 24
Example 2.6 (Representing the XOR Program as a String) . . . 24
2.10 NAND-CIRC Interpreter in NAND-CIRC . . . 25
Theorem 2.13 (Existence of a Bounded Universal Program) . . . 25
Definition 2.26 (Bounded Universal Program) . . . 26
Theorem 2.14 (Efficient bounded universality of NAND-CIRC programs) . . . 26

3 Regular Languages 27
Example 3.1 (XOR as a Parity Function) . . . 27
Example 3.2 (The Multiplication Function) . . . 27
Example 3.3 (Palindrome function) . . . 27
Definition 3.1 (Booleanizing a Function) . . . 27
Example 3.4 (Boolean variant of MULT) . . . 27
3.1 Regular Grammars . . . 28
Definition 3.2 (Regular Grammar) . . . 28
3.2 Regular Expressions . . . 28
Definition 3.3 (Regular Expression) . . . 28
Example 3.5 (Regular Expressions over Binary Alphabet) . . . 28
Definition 3.4 (Matching a RegEx) . . . 29
Example 3.6 (Matching a RegEx) . . . 29
Definition 3.5 (Recognized Language of a RegEx) . . . 29
Example 3.7 (Generated Language of all C Identifiers) . . . 29
Example 3.8 (Numeric Identifiers) . . . 29
Definition 3.6 (Regular Boolean Function) . . . 30
Example 3.9 (Counterexamples to Writing Balanced Parentheses) . . . 30
3.3 Finite Automata . . . 30
Definition 3.7 (Deterministic Finite Automaton) . . . 30
Definition 3.8 (DFA Computability) . . . 30
Definition 3.9 (Recognized Language of a DFA) . . . 30
Definition 3.10 (Nondeterministic Finite Automaton) . . . 31
Definition 3.11 (Recognized Language of an NFA) . . . 31
Definition 3.12 (Operations on Finite Automata) . . . 31
Lemma 3.1 (Union of NFA) . . . 32
Lemma 3.2 (Union of DFA) . . . 32
Lemma 3.3 (Complement of DFA) . . . 32
Example 3.10 (Cannot Flip Accept States in NFA) . . . 32
Lemma 3.4 (Complement of NFA) . . . 33
Theorem 3.5 (Accepting Nothing) . . . 33
Theorem 3.6 (Equivalence of NFA/DFAs) . . . 33
Example 3.11 (Computing a DFA) . . . 33
Example 3.12 (Converting NFA to DFA) . . . 34
Example 3.13 (Converting NFA to DFA) . . . 34
Example 3.14 (NFAs are More Verifiable to Humans) . . . 37
Example 3.15 (Deriving RegEx for Comments from NFA) . . . 38
3.4 Equivalent Forms of Regularity . . . 40

Theorem 3.7 (Equivalence of Regular Language) . . . 40

4 Context Free Languages **41**
4.1 Context Free Grammars . . . 41
Example 4.1 (Arithmetic Function) . . . 41
Definition 4.1 (Context Free) . . . 41
Example 4.2 (A Grammar for Arithmetic Expressions) . . . 41
Definition 4.2 (Ambiguous) . . . 42
Example 4.3 (Ambiguous Context Free Grammar) . . . 42
Example 4.4 (Non-Ambiguous Context-Free Grammar) . . . 42

5 Context Sensitive Languages **44**
Definition 5.1 (Context Sensitive) . . . 44

6 Turing Machines **45**
Definition 6.1 (Turing Machine) . . . 45
Example 6.1 (Turning Machine for Palindromes) . . . 45
Definition 6.2 (Computable Functions) . . . 46
Definition 6.3 (The class **R**) . . . 46
6.1 NAND-TM Programs . . . 47
Definition 6.4 (NAND-TM Programs) . . . 48
Theorem 6.1 (Equivalence of Turing Machines and NAND-TM programs) . . . 49
6.1.1 Uniformity of Computation . . . 49
Definition 6.5 (Uniformity) . . . 49
6.2 RAM Machines and NAND-RAM Programs . . . 50
Definition 6.6 (RAM Machine) . . . 50
Theorem 6.2 (Equivalence of Turing and RAM Machines) . . . 51

7 Turing Completeness and Equivalence **51**
Definition 7.1 (Turing-Complete) . . . 51
Definition 7.2 (Turing Completeness and Equivalence) . . . 52
7.1 Cellular Automata . . . 52
Definition 7.3 (Conway's Game of Life) . . . 52
Theorem 7.1 (Conway's Game of Life Is Turing Complete) . . . 53
7.1.1 One-Dimensional Cellular Automata . . . 53
Definition 7.4 (One-Dimensional Cellular Automaton) . . . 53
Theorem 7.2 (Rule 110 Is Turing Complete) . . . 53
Definition 7.5 (Configuration of Turing Machines) . . . 53
7.2 Lambda Calculus . . . 54
Definition 7.6 (λ expression) . . . 54
7.2.1 Applications . . . 55
Example 7.1 (Addition in the Lambda Calculus) . . . 55
7.2.2 Abstractions . . . 56
7.2.3 Beta Reduction . . . 56
Example 7.2 (Beta Normal Form) . . . 56
7.2.4 Combinators . . . 56
7.2.5 Free and Bound Variables . . . 56
Definition 7.7 (Free Variables of a Lambda Expression) . . . 56
Example 7.3 (Simplifying a Lambda Expression) . . . 57
7.2.6 Booleans as Functions . . . 57

8 Universality **58**
Theorem 8.1 (Universal Turing Machine) . . . 58
Definition 8.1 (String representation of Turing machine) . . . 58

9 Time Complexity **60**
Definition 9.1 (Big-O Notation) . . . 60
Example 9.1 (Simple Runtime Calulation for Polynomials) . . . 60
9.1 Formally Defining Running Time . . . 60
Definition 9.2 (Computable in Turing Machine Time $T(n)$) . . . 60
9.1.1 Polynomial and Exponential Time . . . 61
Definition 9.3 (Decision Problem) . . . 61
Example 9.2 (Examples of Decision Problems) . . . 61
Definition 9.4 (Polynomial and Exponential Time) . . . 61
Lemma 9.1 ($\mathbf{P} \subseteq \mathbf{EXP}$) . . . 61

9.2 Modeling Running Time Using RAM Machines/NAND-RAM . . . 62
Definition 9.5 (Computable in RAM Time $T(n)$) . . . 62
Theorem 9.2 (Relating RAM and Turing machines) . . . 62
Definition 9.6 (Nice Time Bound Function) . . . 63
9.3 Efficient Universal Machine: A NAND-RAM Interpreter in NAND-RAM . . . 63
Theorem 9.3 (Efficient Universality of NAND-RAM) . . . 63
9.4 The Time Hierarchy Theorem . . . 64
Theorem 9.4 (Time Hierarchy Theorem) . . . 64
9.5 Non-Uniform Computation . . . 64

10 Polynomial-Time Reductions 64
10.1 Polynomial-Time Reductions . . . 65
Definition 10.1 (Polynomial-time reductions) . . . 65
10.2 Reducing 3SAT to Zero-One and Quadratic Equations . . . 65
Definition 10.2 (Zero-One Linear Equations Problem) . . . 65
Theorem 10.1 (Hardness of 01 Linear Equations) . . . 65
Theorem 10.2 (Hardness of Quadratic Equations) . . . 66
10.3 Independent Set and Other Graph Problems . . . 66
Definition 10.3 (Independent Set) . . . 66
Theorem 10.3 (Hardness of Independent Set) . . . 66
Definition 10.4 (Vertex Cover) . . . 66
Theorem 10.4 ($3SAT \leq_p VC$) . . . 67
Definition 10.5 (Clique) . . . 67
Theorem 10.5 (Equivalence of CLIQUE and ISET) . . . 67
Definition 10.6 (Dominating Set) . . . 67
10.3.1 Anatomy of a Reduction . . . 68
Theorem 10.6 (Hardness of Longest Path) . . . 68

11 NP, NP Completeness, and Cook-Levin Theorem 68
11.1 The Class NP . . . 68
Definition 11.1 (NP - Nondeterministic Polynomial Time) . . . 68
Theorem 11.1 ($\mathbf{P} \subseteq \mathbf{NP}$) . . . 69
11.2 NP Hard and NP Complete Problems . . . 70
Theorem 11.2 (Cook-Levin Theorem) . . . 70
Definition 11.2 (NP-Hard) . . . 70
11.3 P = NP? . . . 70
11.4 NANDSAT, 3NAND Problems . . . 71
Definition 11.3 (The NANDSAT Problem) . . . 71
Definition 11.4 (The 3NAND Problem) . . . 71
Example 11.1 (A 3NAND Formula) . . . 71
Theorem 11.3 (NP-Completeness of NANDSAT and 3NAND) . . . 71

12 Intractability 72
12.1 Uncomputable Functions . . . 72
Definition 12.1 (The Halting Problem) . . . 72
Theorem 12.1 (HALT Is Not Computable) . . . 72
12.2 Impossibility of General Software Verification . . . 72
Definition 12.2 (Semantic Property) . . . 72
Example 12.1 (A Semantic Property of a Program) . . . 72
Example 12.2 (Two Programs Computing the Same Function) . . . 72
Definition 12.3 (Semantic properties) . . . 73
Theorem 12.2 (Rice's Theorem) . . . 73
Corollary 12.3 (COMPUTES-PARITY Is Uncomputable) . . . 73

13 Probabilistic Computation 74
13.1 Finding Approximately Good Maximum Cuts . . . 74
Theorem 13.1 (Approximating Max Cut) . . . 74
13.1.1 Amplifying the success of randomized algorithms . . . 74
Lemma 13.2 (A Random Cut Cuts Half the Edges) . . . 74
13.1.2 Success Amplification . . . 75
Lemma 13.3 (Randomized Max-Cut Approximation) . . . 75
13.1.3 Two-sided Amplification . . . 75
Theorem 13.4 (Amplification of One-Sided Error) . . . 75
13.1.4 Solving SAT through Randomization . . . 76
Theorem 13.5 (WalkSAT simple analysis) . . . 76

13.1.5 Bipartite Matching . . . 76
Definition 13.1 (Bipartite Graph) . . . 76
Lemma 13.6 (Matching polynomial) . . . 77
Theorem 13.7 (Schwartz-Zippel Lemma) . . . 78
13.2 Modeling Randomized Computation . . . 78
Definition 13.2 (The class **BPP**) . . . 79
Definition 13.3 (Alternative characterization of **BPP**) . . . 79
Theorem 13.8 (Sipser-Gacs Theorem) . . . 79
13.2.1 Success Amplification of two-sided error algorithms . . . 79
Theorem 13.9 (Amplification) . . . 79
13.2.2 BPP and NP Completeness . . . 80
Theorem 13.10 (NP-Hardness inside BPP Collapses the Hierarchy) . . . 80
13.3 The Power of Randomization . . . 80
Theorem 13.11 (Simulating randomized algorithms in exponential time) . . . 80

This covers computability theory, complexity theory, and automata theory.

1 Formal Languages

1.1 Alphabets, Strings, and Languages

Definition 1.1 (Alphabet)

An **alphabet** Σ is a set.

Definition 1.2 (Kleene Star)

Given a set Σ, we define

$$\Sigma^* \coloneqq \bigcup_{n=0}^{\infty} \Sigma^n \tag{1}$$

Each element of Σ^* is called a **word** or a **string**, and a subset of Σ^* is called an **expression**. The **empty word**, denoted ϵ, is the unique string of length 0.

Now let's define some operations.

Definition 1.3 (Concatenation)

Given two words $u, v \in \Sigma^*$, the **concatenation** $uv = u \cdot v$ is the word formed by appending the sequence of symbols in v to the sequence of symbols in u.

Note that $(\Sigma^*, \cdot, \epsilon)$ is a monoid, where ϵ represents the empty word. Some strings (programs) are legal while others are not. Therefore, we can define the set of all valid programs as simply as a subset of Σ^*.

Definition 1.4 (Formal Language)

A **formal language** over alphabet Σ is a subset $L \subset \Sigma^*$.
1. A word $w \in \Sigma^*$ is **well-formed** if $w \in L$.
2. An expression $E \subset \Sigma^*$ is **well-formed** if $E \subset L$.

We can do operations on languages.

Definition 1.5 (Product of Formal Languages)

Given two formal languages $L_1, L_2 \subset \Sigma^*$, their product is defined

$$L_1 L_2 \coloneqq \{uv \mid u \in L_1, v \in L_2\} \tag{2}$$

1.2 Decision Problems as Languages

Decision problems as languages

1.3 Encodings

Encodings of mathematical objects as strings

1.4 Formal Grammars

Generally, a subset of Σ^* doesn't give us much structure, so we would like to define some way to pick the subset L out. We can do this with *grammars*.

Definition 1.6 (Formal Grammar)

A **formal grammar** is a 4-tuple $G = (V, \Sigma, R, S)$ consisting of the following.

1. *Non-terminal Symbols.* V is a finite set consisting of **non-terminal symbols**, also known as **variables**.
2. *Terminal Symbols.* Σ is a finite set—disjoint from V—consisting of **terminals**.
3. *Production Rule.* R is a relation, i.e. is a finite subset of

$$(V \cup \Sigma)^* V (V \cup \Sigma)^* \times (V \cup \Sigma)^* \tag{3}$$

   where the LHS represents the product of the languages. We usually write the relation as $\alpha \to \beta$.[a]
4. *Start Symbol.* $S \in V$ is the initial variable from which derivation begins.

[a] In math, this is usually written $\alpha R \beta$, or $\alpha \sim \beta$ if it is an equivalence relation.

We can think of the production rule as a set of transformations—formally called *derivations*—that you can apply to a word.

Definition 1.7 (Derivation)

Let $G = (V, \Sigma, R, S)$ be a formal grammar. We define a binary relation $\Rightarrow$ on the set of all strings $(V \cup \Sigma)^*$ as follows:

1. A string u **directly derives** v, written $u \Rightarrow v$, if there exists strings $\phi, \psi \in (V \cup \Sigma)^*$ and a production rule $(\alpha \to \beta) \in R$ such that

$$u = \phi\alpha\psi, \qquad v = \phi\beta\psi \tag{4}$$

   In other words, we say a word w is derived from v, written $v \Rightarrow w$, if w can be obtained by replacing a part of v according to a rule in R.
2. We say u **derives** v[a], written $u \overset{*}{\Rightarrow} v$, if u can be directly derived into v in 0 or more steps. The relation $\overset{*}{\Rightarrow}$, called the **reflexive transitive closure** of $\Rightarrow$, is defined as follows. $u \overset{*}{\Rightarrow} v$ if
   (a) $u = v$, or
   (b) There exists a finite sequence of strings $u = w_0, w_1, \ldots, w_n = v$ such that $w_i \Rightarrow w_{i+1}$ for all $0 \leq i < n$.

[a] We also say that v is in the transitive closure of u.

Definition 1.8 (Language Derived From Grammar)

The **language derived from** G, denoted $L(G)$, is the set of all terminal strings that can be derived from S using the rules in R. It is defined:

$$L(G) \coloneqq \{w \in \Sigma^* \mid S \Rightarrow^* w\} \tag{5}$$

Example 1.1 (A Grammar for Additive Expressions)

Let $V = \{E\}$ be the non-terminals, and $\{+, -, \texttt{num}\}$ be our terminals, which come from our lexer. Then, our production rules can look something like.

1. $E \to E + E$.
2. $E \to E - E$.
3. $E \to \texttt{num}$.

So basically, if we have some word, then we can replace any E in the word by any of the three choices on the right hand side. For example, we can do the following sequence of derivations.

$$\begin{aligned} E &\Rightarrow E + E && (6)\\ &\Rightarrow E - E + E && (7)\\ &\Rightarrow E - \texttt{num} + E && (8)\\ &\Rightarrow E + E - \texttt{num} + E && (9) \end{aligned}$$

You can keep doing this until there is no more production rules you can apply. For context free grammars, you basically do this until there are only terminals left, e.g. $\texttt{num} + \texttt{num} - \texttt{num} + \texttt{num}$.

You can track the derivations used to go from the base expression to the sequence of terminals. This is perfectly represented by a tree.

Definition 1.9 (Parse Tree)

Given a formal grammar $G = (V, \Sigma, R, S)$, a **parse tree** is an ordered tree where

1. the root is the start symbol S.
2. each internal node is a nonterminal symbol.
3. each leaf is either a terminal symbol or ϵ
4. if an internal node is labeled by a nonterminal A, and its children from left to right are labeled $X_1, \dots, X_k$, then the grammar must contain the production

$$A \Rightarrow X_1 X_2 \dots X_k \tag{10}$$

The **yield** of a parse tree is the string obtained by reading the leaves from left to right.

Definition 1.10 (Transitive Closure)

Definition 1.11 (Language Generated by Grammar)

Given grammar $G = (V, \Sigma, R, S)$, the language $L(G)$ is the set of all terminal strings that can be derived from the start symbol S.

$$L(G) \coloneqq \{w \in \Sigma^* \mid S \Rightarrow^* w\} \tag{11}$$

Therefore, you can basically think of the "complexity" or size of a language L as being determined by the "complexity" of the generating grammar G. Usually, the alphabet is kept fixed, and the main contributor to the complexity are the production rules R. Depending on how much we restrict R, we get different levels of languages in the *Chomsky Hierarchy*.

Definition 1.12 (Unrestricted)

An **unrestricted language** L is a language that can be generated by some grammar G.

As the name suggests, there will be languages which are restricted. The hierarchy of these restricted languages is called the **Chomsky hierarchy**.

2 Finite Boolean Computation

Given alphabet Σ, functions of the form $f : \Sigma^n \to \Sigma^m$ with a finite domain and codomain are called *finite functions*, and if $\Sigma = \{0, 1\}$, then it is said to be a *boolean function*. Mathematical functions are an abstraction, and our goal is to look for physical [1] or lexical models that can *compute* these functions, i.e. have some representation that essentially behaves indistinguishably from the function. The two main categories of representations that we will focus on are circuits and straight-line programs.

Considering the set of all finite functions $f : \{0, 1\}^n \to \{0, 1\}^m$ is too unstructured. Taking inspiration from math, we wish to find nice *decompositions* of functions f into simpler ones. The most elementary functions in boolean logic are the AND, OR, and NOT functions.

Definition 2.1 (AND Function)

The AND function is defined

$$\text{AND} : \{0,1\}^2 \longrightarrow \{0,1\}, \qquad \text{AND}(a,b) = a \wedge b = \begin{cases} 1 & a = b = 1 \\ 0 & \text{else} \end{cases} \tag{12}$$

Definition 2.2 (OR Function)

The OR function is defined

$$\text{OR} : \{0,1\}^2 \longrightarrow \{0,1\}, \qquad \text{OR}(a,b) = a \vee b = \begin{cases} 0 & a = b = 0 \\ 1 & \text{else} \end{cases} \tag{13}$$

Definition 2.3 (NOT Function)

The NOT function is defined

$$\text{NOT} : \{0,1\} \longrightarrow \{0,1\}, \qquad \text{NOT}(a) = \neg a = \begin{cases} 0 & a = 1 \\ 1 & a = 0 \end{cases} \tag{14}$$

Great, so we have developed three rudimentary finite functions—one unary and two binary. We can use these functions to build new functions by composing them. Mathematically, we can write out the symbols as such, for example.

$$f(x, y) = \text{AND}(\text{NOT}(x), \text{OR}(x, y)) \tag{15}$$

Many functions can be created when composing these extremely simple functions.

Example 2.1 (Majority Function)

Consider the function that outputs whatever the majority bit value is amongst its 3 inputs.

$$\text{MAJ} : \{0,1\}^3 \longrightarrow \{0,1\}, \qquad \text{MAJ}(x) = \begin{cases} 1 & x_0 + x_1 + x_2 \geq 2 \\ 0 & \text{else} \end{cases} \tag{16}$$

[1] as in, they can be realized by physical systems that we will build on in my computer architecture notes, through they are still theoretical

Since the OR of three conditions c_0, c_1, c_2 can be written as $\text{OR}(c_0, \text{OR}(c_1, c_2))$, we can now translate this function into a formula as follows:

$$\begin{aligned} \text{MAJ}(x_0, x_1, x_2) &= \text{OR}\left(\text{AND}(x_0, x_1), \text{OR}(\text{AND}(x_1, x_2), \text{AND}(x_0, x_2))\right) && (17) \\ &= \big((x_0 \wedge x_1) \vee (x_1 \wedge x_2)\big) \vee (x_0 \wedge x_2) && (18) \end{aligned}$$

A natural question to ask is whether *any* finite function can be modeled as a composition of these three functions.

Definition 2.4 (Universal Operation Set)

A set of boolean functions $\mathcal{F}$ is said to be **universal** if any finite boolean function $f : \{0,1\}^n \to \{0,1\}^m$ can be written as a composition of functions in $\mathcal{F}$.

Theorem 2.1 (AON Functions are Universal)

$\{\text{AND}, \text{OR}, \text{NOT}\}$ is a universal function set. In fact, any function $f : \{0,1\}^n \longrightarrow \{0,1\}^m$ can be written as a composition of at most $cm2^n$ AON functions, for some constant $c > 0$.

Proof.

This may not be so surprising actually. After all, a finite function $f : \{0,1\}^n \longrightarrow \{0,1\}^m$ can be represented by simply the list of its outputs for each one of the 2^n input values. So it makes sense that we could write a composition of similar size to compute it.

This simplifies things. If we can make a computational model that can simulate the AND, OR, NOT functions, this model will be able to compute every function.

2.1 Circuits

We introduce our first physical model in the form of circuits. Why a circuit? As we will see later in my computer architecture notes, a nice way to physically realize such functions are through transistors, which are similar to gates.

Definition 2.5 (Boolean Circuit)

Let n, m, s be positive integers with $s \geq m$. A **Boolean $\mathcal{G}$-circuit** with n inputs, m outputs, and s gates, is a labeled directed acyclic graph (DAG) $C = (V, E)$ with $|V| = s + n$ vertices satisfying the following properties:

1. The n **inputs** refer to vertices that have no in-neighbors. Each input has at least one out-neighbor and are labeled either as $X[i]$ or not at all.
2. The other s vertices are known as **gates** $g \in \mathcal{G}$. The size of C is s.
3. The m **outputs** refer to vertices v that have an out-neighbor u that is not an in-neighbor of any other node. The out-neighbors are labeled either as $Y[j]$ or not at all.

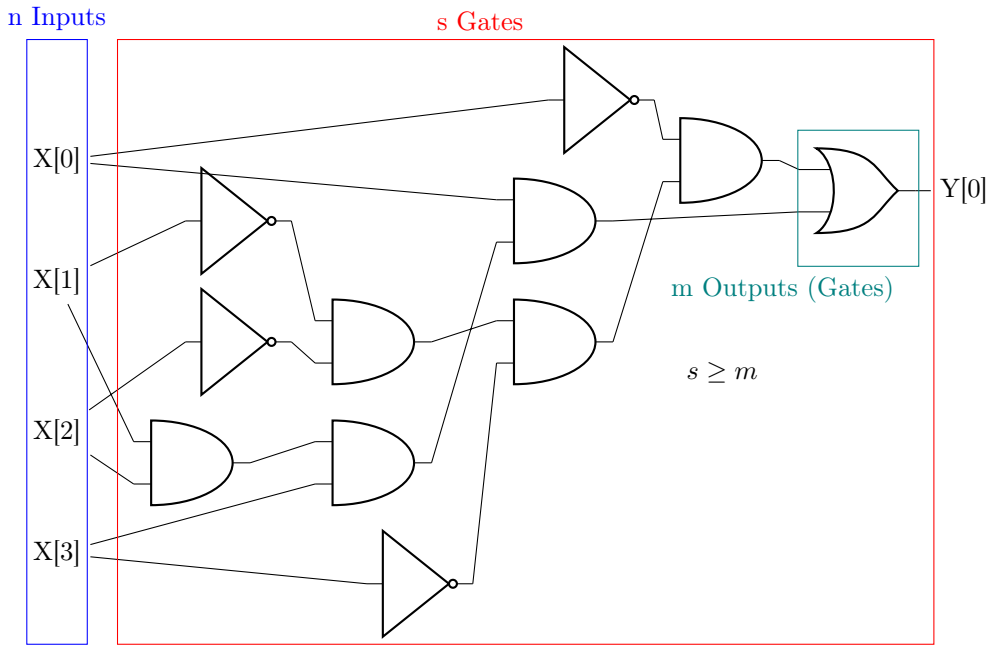


Figure 1

Constructing a circuit is similar to composing elementary functions, and the analogue of such functions are *logic gates*.[2] Let's define some elementary logic gates. Since we have seen that AON is universal, we can define the analogue of the three to get another universal gate set.

Definition 2.6 (NOT Gate)

A **NOT gate** is a physical representation of the NOT function.

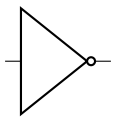


Figure 2: A helpful hint to remember that this is NOT. Think of the triangle shape as "doing nothing" and pay attention to the circle at the tip, which represents negation.

Definition 2.7 (AND Gate)

A **AND gate** is a physical representation of the AND function.

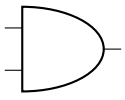


Figure 3: A helpful hint to remember that this is AND. Think of the D-shape as requiring both inputs to be 1 for the output to be 1.

[2]Note that gates are *not* the same as functions, but a representation of them.

Definition 2.8 (OR Gate)

A **OR gate** is a physical representation of the OR function.

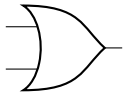


Figure 4: A helpful hint to remember that this is OR. Think of the rounded shape as requiring at least one input to be 1 for the output to be 1.

A circuit $C : \{0,1\}^n \to \{0,1\}^m$ can be thought of as a function from the way it behaves on an input $x \in \{0,1\}^n$. Analogously to our mathematical model, we can define the representation power of circuits.

Definition 2.9 (Computability of Circuits)

Let C be a $\mathcal{G}$-circuit with n inputs and m outputs, and let $f : \{0,1\}^n \to \{0,1\}^m$ be an arbitrary function. C is said to **compute** f if $C = f$ as functions.

Definition 2.10 (Universal Gate Set)

A set of gates $\mathcal{G}$ is said to be a **universal gate set** if every finite function f is computable by a $\mathcal{G}$-circuit.

Definition 2.11 (AON-CIRC)

An **And/Or/Not circuit**, abbreviated as **AON-CIRC**, is a circuit C made of AND, OR, and NOT gates.

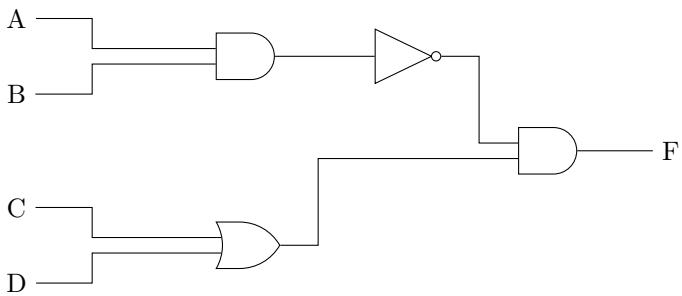
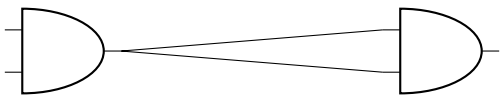


Figure 5: An example of an AON circuit. Circuits have wires connecting into (from the left) and out (from the right) of each gate. Wires that are not connected to any gate from the left represent inputs to the circuit and from the right represent outputs of the circuit.

Note that parallel edges are allowed. Having parallel edges means that an AND or OR gate u can have both its in-neighbors be the same gate v. Since $AND(a,a) = OR(a,a) = a$ for every $a \in \{0,1\}$, such parallel gates don't help in computing new values in circuits with AND/OR/NOT gates.



It is immediate that AON-CIRC is universal by construction.

Theorem 2.2 (AON-CIRC is Universal)

$\{\text{AND}, \text{OR}, \text{NOT}\}$ is a universal gate set, i.e. every finite function f can be computed by an AON-CIRC.

Proof.

Example 2.2 (All Equals Function)

Let us define the function ALLEQ : $\{0,1\}^4 \longrightarrow \{0,1\}$ to be the function that on input $x \in \{0,1\}^4$ outputs 1 if and only if $x_0 = x_1 = x_2 = x_3$.

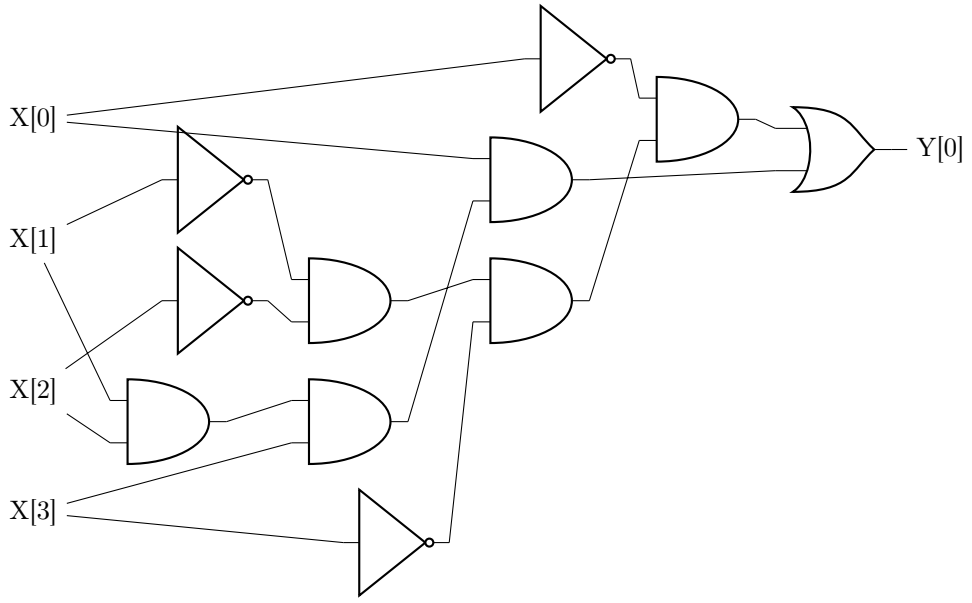


Figure 6: The Boolean circuit for computing ALLEQ.

2.2 Straight Line Programs

Our second computational model will be *lexical*—rather than a physical—one. This is more closely related to coding languages and has the advantage that it is often easier to work with for humans.

Definition 2.12 (Straight Line Program)

Let $\mathcal{F} = \{f_0, f_1, ..., f_{t-1}\}$ be a finite collection of Boolean operators[a] $f_i : \{0,1\}^k \longrightarrow \{0,1\}$.
A **$\mathcal{F}$-straight line program** is a program P composed of a finite set of operators $f \in \mathcal{F}$. It consists of a finite sequence of lines, each of which assigns to some variable the result of applying some $f_i \in \mathcal{F}$ to k_i other variables.

$$v = f(w, \dots, u), \qquad f \in \mathcal{F} \tag{19}$$

In every line, the variables on the right-hand side of the assignment operators must either be input variables or variables that have already been assigned a value.[b] `X[i]` and `Y[j]` denotes the input and output variables.

```
foo1 = f(bar,blah)
foo2 = g(bar,blah)
foo3 = h(bar)
...
```

Figure 7: An example of a straight line program.

[a]We can think of this as some abstraction from a physical implementation of a function.
[b]It is called a straight-line program since it contains no loops or branching (e.g. if/then statements).

Again, a straight line program can be thought of as a function.

Definition 2.13 (Computability of Straight-Line Programs)

Let P be a $\mathcal{F}$-straight line program with n inputs and m outputs, and let $f : \{0,1\}^n \to \{0,1\}^m$ be an arbitrary function. P is said to **compute** f if $P = f$ as functions.

Definition 2.14 (Universal Operator Set)

A set of operators $\mathcal{F}$ is said to be a **universal gate set** if every finite function f is computable by a $\mathcal{F}$-circuit.

Definition 2.15 (AON Straight Line Program)

The **AON straight line program**, abbreviated **AON-SLP**, is a straight line prgoram P with the AND, OR, and NOT operators.

Again, it is immediate that AON straight line programs are universal since it models AON.

Theorem 2.3 (AON-SLP)

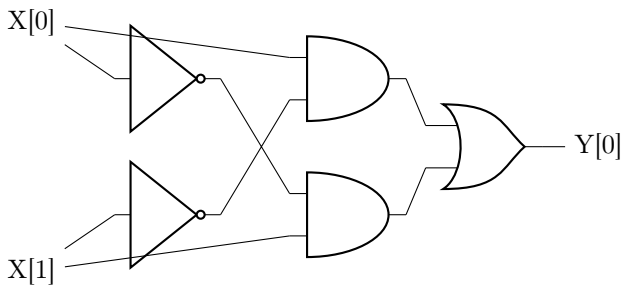
$\{\text{AND}, \text{OR}, \text{NOT}\}$ is a universal operator set, i.e. every finite function f can be computed by an AON-SLP.

Example 2.3 (XOR Function)

Let the XOR function be defined

$$XOR : \{0,1\}^2 \longrightarrow \{0,1\},\ XOR(a,b) = a + b \pmod 2 \tag{20}$$

The Boolean circuit for computing $XOR : \{0,1\}^2 \longrightarrow \{0,1\}$ is:



This can be computed with the straight-line algorithm as such. Given (a, b) as inputs, we have $w_1 = AND(a, b), w_2 = NOT(w_1)$, and $w_3 = OR(a, b)$. Then the algorithm returns $AND(w_2, w_3)$. In Python, this can be programmed:

```
def AND(a, b): return a*b
def OR(a, b): return 1-(1-a)*(1-b)
def NOT(a): return 1-a

def XOR(a, b):
    w1 = AND(a, b)
    w2 = NOT(w1)
    w3 = OR(a,b)
    return AND(w2, w3)

print([f"XOR({a},{b})={XOR(a,b)}" for a in [0,1] for b in [0,1]])
# ['XOR(0,0)=0', 'XOR(0,1)=1', 'XOR(1,0)=1', 'XOR(1,1)=0']
```

2.3 NAND Circuits and Other Gate Sets

We have seen that both AON-CIRC and AON-SLP can both compute the same set of all finite functions, i.e. they are *equivalent in power*.

Definition 2.16 (NAND Gate)

A **NAND gate** is defined as follows.[a]

$$\text{NAND} : \{0,1\}^2 \longrightarrow \{0,1\}, \qquad \text{NAND}(a,b) = \overline{a \wedge b} = \begin{cases} 0 & a = b = 1 \\ 1 & \text{else} \end{cases} \tag{21}$$

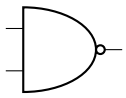


Figure 8: A helpful hint to remember that this is NAND. Notice the D-shape of the AND gate with the circle at the output representing negation (NOT).

[a] NAND is really the composition of the NOT and AND functions; that is, $\text{NAND}(a,b) = (\text{NOT} \circ \text{AND})(a,b)$.

Theorem 2.4 (NAND is Universal)

The NAND function is universal. It follows that NAND gates and operators are universal.

Proof. We can see that, using double negation,

$$\begin{aligned}
\text{NOT}(a) &= \text{NOT}(\text{AND}(a,a)) && (22)\\
&= \text{NAND}(a,a) && (23)\\
\text{AND}(a,b) &= \text{NOT}(\text{NOT}(\text{AND}(a,b))) && (24)\\
&= \text{NOT}(\text{NAND}(a,b)) && (25)\\
&= \text{NAND}(\text{NAND}(a,b),\text{NAND}(a,b)) && (26)\\
\text{OR}(a,b) &= \text{NOT}(\text{AND}(\text{NOT}(a),\text{NOT}(b))) && (27)\\
&= \text{NOT}(\text{AND}(\text{NAND}(a,a),\text{NAND}(b,b))) && (28)\\
&= \text{NAND}(\text{NAND}(a,a),\text{NAND}(b,b)) && (29)
\end{aligned}$$

Just as we have defined the AON-CIRC program, we can define the notion of computation by a NAND-CIRC program in the natural way.

Theorem 2.5 (Equivalence Between Circuits and Straight Line Programs)

AON/NAND circuits and AON/NAND straight-line programs are equivalent in power. Furthermore, for every sufficiently large s, n, m and $f : \{0,1\}^n \longrightarrow \{0,1\}^m$, the following conditions are all equivalent to one another:

1. f can be computed by a Boolean circuit (with $\wedge, \vee, \neg$ gates) of at most $O(s)$ gates.
2. f can be computed by an AON-CIRC straight-line program of at most $O(s)$ lines
3. f can be computed by a NAND circuit of at most $O(s)$ gates.
4. f can be computed by a NAND-CIRC straight-line program of at most $O(s)$ lines.

By $O(s)$, we mean that the bound is at most $c \cdot s$, where c is a constant that is independent of n. For example, if f can be computed by a Boolean circuit of s gates, then it can be computed by a NAND-CIRC program of at most $3s$ lines, and if f can be computed by a NAND circuit of s gates, then it can be computed by an AON-CIRC program of at most $2s$ lines.

Definition 2.17 (NOR Gate)

A **NOR gate** is defined as follows.[a]

$$\text{NOR} : \{0,1\}^2 \longrightarrow \{0,1\}, \qquad \text{NOR}(a,b) = \overline{a \vee b} = \begin{cases} 1 & a = b = 0 \\ 0 & \text{else} \end{cases} \tag{30}$$

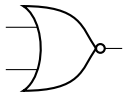


Figure 9: A helpful hint to remember that this is NOR. Notice the rounded shape of the OR gate with the circle at the output representing negation (NOT).

[a] NOR is really the composition of the NOT and OR functions; that is, $\text{NOR}(a,b) = (\text{NOT} \circ \text{OR})(a,b)$.

Definition 2.18 (XOR Gate)

A **XOR gate** is defined as follows.

$$\text{XOR} : \{0,1\}^2 \longrightarrow \{0,1\}, \qquad \text{XOR}(a,b) = a \oplus b = \begin{cases} 0 & a = b \\ 1 & a \neq b \end{cases} \tag{31}$$

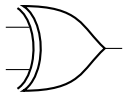


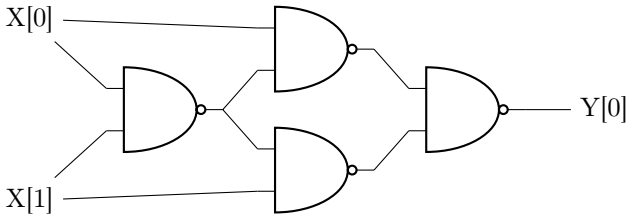
Figure 10: A helpful hint to remember that this is XOR. Notice the additional curve on the OR gate shape, indicating that exactly one input must be 1 for the output to be 1.

Example 2.4 (XOR from NAND)

XOR can be expressed in terms of other logic gates as follows:

$$\text{XOR}(a,b) = (a \wedge \neg b) \vee (\neg a \wedge b) \tag{32}$$

We can create a NAND circuit of the XOR function that maps $x_0, x_1 \in \{0,1\}$ to $x_0 + x_1 \pmod 2$.



There are some sets $\mathcal{F}$ that are more restricted in power. For example, it can be shown that if we use only AND or OR gates (without NOT), then we do not get an equivalent model of computation.

2.4 Conditionals

Just as we have built the AND, OR, and NOT gates with the NAND gate, we can implement more complex features using our basic building blocks, and then use these new features themselves as building blocks for even more sophisticated features. This is known as **syntactic sugar**, since we are not modifying the underlying programming model itself, but rather we merely implement new features by syntactically transforming a program that uses such features into one that doesn't. It makes the language "sweeter" for human use: things can be expressed more clearly, more concisely, or in an alternative style that some may prefer.

In computer programming, we can define and then execute **procedures** or **subroutines**, which are often known as *functions*.

Example 2.5 (Syntactic Sugar to Define Majority Function)

We can use syntactic sugar to compute the majority function MAJ as follows, by first defining the procedures NOT, AND, and OR.

```
def MAJ(a,b,c):
  and1 = AND(a,b)
  and2 = AND(a,c)
```

```
    and3 = AND(b,c)
    or1 = OR(and1,and2)
    return OR(or1,and3)

print(MAJ(0,1,1)) # 1
```

Note that compared to writing out the full Boolean circuit without any syntactic sugar, one with sugar will can be much simpler. It's the difference between having access to only NAND, or all of NAND, AND, OR, NOT.

Definition 2.19 (NAND-CIRC-PROC)

We call these the programming language NAND-CIRC augmented with the syntax above (for defining procedures) a **NAND-CIRC-PROC** program. Note that NAND-CIRC-PROC only allows *non-recursive* procedures (that is, procedures that do not take in its return value as its argument).

We can define conditional (if/then) statements using NAND operators. The idea is to compute the function $IF : \{0,1\}^3 \longrightarrow \{0,1\}$ such that $IF(a,b,c)$ equals b if $a = 1$ and c if $a = 0$.

Definition 2.20 (Multiplexor Gate)

A **Multiplexor gate** (MUX) is defined as follows.

$$\text{MUX} : \{0,1\}^3 \longrightarrow \{0,1\}, \qquad \text{MUX}(a,b,s) = \begin{cases} a & s = 0 \\ b & s = 1 \end{cases} \tag{33}$$

A multiplexor can be expressed using basic logic gates as follows:

$$\text{MUX}(a,b,s) = (a \wedge \neg s) \vee (b \wedge s) \tag{34}$$

Definition 2.21 (Demultiplexor Gate)

A **Demultiplexor gate** (DEMUX) is defined as follows.

$$\text{DEMUX} : \{0,1\}^2 \longrightarrow \{0,1\}^2, \qquad \text{DEMUX}(d,s) = (y_0, y_1) \text{ where } \begin{cases} y_0 = d \wedge \neg s \\ y_1 = d \wedge s \end{cases} \tag{35}$$

A demultiplexor routes the input d to one of two outputs based on the select signal s.

Definition 2.22 (Conditional Statement)

The IF function can be implemented from NANDs as follows:

```
def IF(cond, a, b);
  notcond = NAND(cond, cond)
  temp = NAND(b, notcond)
  temp1 = NAND(a, cond)
  return NAND(temp, temp1)
```

The IF function is also known as a multiplexing function, since *cond* can be thought of as a switch that controls whether the output is connected to a or b.

Proof.

Definition 2.23 (NAND-CIRC-IF)

Let NAND-CIRC-IF be the programming language NAND-CIRC augmented with `if/then/else` statements for allowing code to be conditionally executed based on whether a variable is equal to 0 or 1.

Theorem 2.6 (Removing Syntactic Sugar from NAND-CIRC-IF)

For every NAND-CIRC-IF program P, there exists a standard (i.e. "sugar-free") NAND- CIRC program P' that computes the same function as P.

Theorem 2.7 (Constant Plus Multiplexor is Universal)

Let $\mathcal{F} = \{IF, ZERO, ONE\}$ where

$$ZERO : \{0,1\} \longrightarrow \{0\}, \;\; ONE : \{0,1\} \longrightarrow \{1\} \tag{36}$$

are the constant zero and one functions, and

$$IF : \{0,1\}^3 \longrightarrow \{0,1\}, \; IF(a,b,c) = \begin{cases} b & a = 1 \\ c & else \end{cases} \tag{37}$$

Then, $\mathcal{F}$ is universal.

Proof. We can use the following formula to compute NAND:

$$NAND(a,b) = IF\big(a, IF(b, ZERO, ONE), ONE\big) \tag{38}$$

2.5 Arithmetic

We can write the integer addition function as follows:

```
def ADD(A,B):
  Result = [0]*(n+1)
  Carry = [0]*(n+1)
  Carry[0] = zero(A[0])
  for i in range(n):
    Result[i] = XOR(Carry[i],XOR(A[i],B[i]))
    Carry[i+1] = MAJ(Carry[i],A[i],B[i]) Result[n] = Carry[n]
  return Result

ADD([1,1,1,0,0],[1,0,0,0,0]) # [0, 0, 0, 1, 0, 0]
```

where `zero` is the zero function, and `MAJ, XOR` correspond to the majority and XOR functions respectively. Note that in here, n is a *fixed integer* and so for every such n, `ADD` is a *finite* function that takes as input $2n$ bits and outputs $n+1$ bits. Note that the `for` loop isn't anything fancy at all; it is just shorthand

notation of simply repeating the code n times. By expanding out all the features, for every value of n we can translate the above program into a standard ("sugar-free") NAND-CIRC program. Note that the sugar free NAND-CIRC program to adding two-digit binary numbers consists of 43 lines of code, with a Boolean circuit of 15 layers.

We can in fact prove the following theorem that gives an upper bound on the addition algorithm.

Theorem 2.8 (Addition using NAND-CIRC programs)

For every $n \in \mathbb{N}$, let

$$ADD_n : \{0,1\}^{2n} \longrightarrow \{0,1\}^{n+1} \tag{39}$$

be the function that, given $x, x' \in \{0,1\}^n$, computes the representation of the sum of the numbers that x and x' represent. Then, for every n there is a NAND-CIRC program to compute ADD_n with at most $9n$ lines.

Once we have addition, we can use grade-school algorithm of multiplication to obtain multiplication as well.

Theorem 2.9 (Muliplication using NAND-CIRC programs)

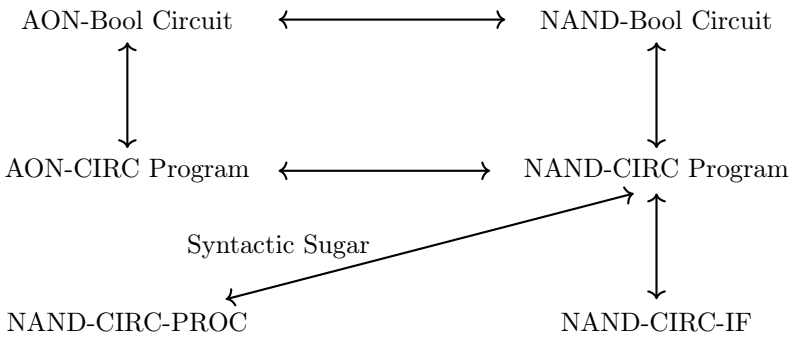
For every n, let

$$MULT_n : \{0,1\}^{2n} \longrightarrow \{0,1\}^{2n} \tag{40}$$

be the function that, given $x, x' \in \{0,1\}^n$, computes the representation of the product of the numbers that x and x' represent. Then, there is a constant c such that for every n, there is a NAND-CIRC program of at most cn^2 that computes the function $MULT_n$.[a]

[a] As we have seen in DSA, *Karatsuba's algorithm* allows us to actually compute that there is a NAND-CIRC program of $O(n^{\log_2 3})$ lines to compute $MULT_n$.

We can summarize the equivalence of these models below:



2.6 Code as Data, Data as Code

A program is simply a sequence of symbols, each of which can be encoded in binary using (for example) the ASCII standard. Therefore, we can represent every NAND-CIRC program (and hence also every Boolean circuit) as a binary string. This means that we can treat circuits or NAND-CIRC programs both as instructions to carrying computation and also as *data* that could potentially be used as *inputs* to other computations. That is, **a program is a piece of text, and so it can be fed as input to other programs**.

Definition 2.24 (The Class $SIZE_{n,m}(s)$)

For every $n, m \in \{1, 2, ..., 2s\}$, let $SIZE_{n,m}(s)$ denote the set of all functions $f : \{0,1\}^n \longrightarrow \{0,1\}^m$ such that $f \in SIZE(s)$. We denote $SIZE_n(s)$ to be just $SIZE_{n,1}(s)$. For every integer $s \geq 1$, we let

$$SIZE(s) = \bigcup_{n,m \leq 2s} SIZE_{n,m}(s) \tag{41}$$

be the set of all functions f that can be computed by NAND circuits of at most s gates (or equivalently, by NAND-CIRC programs of at most s lines).

2.7 Representing Programs as Strings

We can represent programs or circuits as strings in many ways. For example, since Boolean circuits are labeled directed acyclic graphs, we can use the *adjacency matrix* representations. A simpler way is to just interpret the program as a sequence of letters and symbols. For example, the NAND-CIRC program P:

```
temp_0 = NAND(X[0],X[1])
temp_1 = NAND(X[0],temp_0)
temp_2 = NAND(X[1],temp_0)
Y[0] = NAND(temp_1,temp_2)
```

is simply a string of 107 symbols which include lower and upper case letters, digits, the underscore character, equality signs, punctuation marks, space, and the "new line" markers, all of which can be encoded in ASCII. Since every symbol can be encoded as a string of 7 bits using the ASCII encoding, the program P can be encoded as a string of length $7 \cdot 107 = 749$ bits. Therefore, we can prove that *every* NAND-CIRC program can be represented as a string in $\{0,1\}^*$.

Furthermore, since the names of the working variables of a NAND-CIRC program do not affect its functionality, we can always transform a program to have the form of P', where all variables apart from the inputs and outputs, have the form $\texttt{temp}_0$, $\texttt{temp}_1$, ... Moreover, if the program has s, lines, then we will never need to use an index larger than $3s$ (since each line involves at most three variables), and similarly, the indices of the input and output variables will all be at most $3s$. Since a number between 0 and $3s$ can be expressed using at most $\lceil \log_{10}(3s+1) \rceil = O(\log s)$ digits, each line in the program (which has the form `foo = NAND(bar, blah)`), can be represented using $O(1) + O(\log s) = O(\log s)$ symbols, each of which can be represented by 7 bits. This results in the following theorem

Theorem 2.10 (Representing programs as strings)

There is a constant c such that for $f \in SIZE(s)$, there exists a program P computing f whose string representation has length at most $cs \log s$.

2.8 Counting Programs

We can actually see that the number of programs of certain length is bounded by the number of strings that represent them.

Theorem 2.11 (Counting programs)

For every $s \in \mathbb{N}$,

$$|SIZE(s)| \leq 2^{O(s \log s)} \tag{42}$$

That is, there are at most $2^{O(s \log s)}$ functions computed by NAND-CIRC programs of at most s lines. This gives a limitation on NAND-CIRC programs running on at most a given number of s lines.

Note that a function mapping $\{0,1\}^2 \longrightarrow \{0,1\}$ can be identified with a table of its four values on the inputs 00, 01, 10, 11. A function mapping $\{0,1\}^3 \longrightarrow \{0,1\}$ can be identified with the table of its 8 values on the inputs 000, 001, 010, 011, 100, 101, 110, 111. More generally, every function

$$F : \{0,1\}^n \longrightarrow \{0,1\} \tag{43}$$

is equal to the number of such tables which is 2^{2^n}. Note that this is a *double exponential* in n, and hence even form small values of n (e.g. $n = 10$), the number of functions from $\{0,1\}^n \longrightarrow \{0,1\}$ is large.

Theorem 2.12 (Counting argument lower bound)

The shortest NAND-CIRC program to compute $f : \{0,1\}^n \longrightarrow \{0,1\}$ requires more than $\delta \cdot 2^n/n$ lines. That is, there exists a constant $\delta > 0$ such that for every sufficiently large n, there exists $f : \{0,1\}^n \longrightarrow \{0,1\}$ such that $f \notin SIZE\left(\frac{\delta 2^n}{n}\right)$. The constant δ can be proven to be arbitrarily close to $\frac{1}{2}$.

We already know that every function mapping $\{0,1\}^n$ to $\{0,1\}$ can be computed by an $O(2^n/n)$ line program. The previous theorem shows that some functions do require an astronomical number of lines to compute. That is, **some functions $f : \{0,1\}^n \longrightarrow \{0,1\}$ cannot be computed by a Boolean circuit using fewer than exponential (in n) number of gates**.

2.9 Tuples Representation

ASCII is a fine representation of programs, but we can do better. That is, give a NAND-CIRC program with lines of the form

```
blah = NAND(baz, boo)
```

We can encode each line as the triple `(blah, baz, boo)`. Furthermore, we can associate each variable with a number and encode the line with the 3-tuple (i, j, k). Expanding on this, we can associate every variable with a number in the set

$$[t] = \{0, 1, 2, ..., t-1\} \tag{44}$$

where the first n numbers $\{0, ..., n-1\}$ correspond to input variable, the last m numbers $\{t-m, ..., t-1\}$ correspond to the output variables, and the intermediate numbers $\{n, ..., t-m-1\}$ correspond to the remaining variables.

Definition 2.25 (List of tuples representation)

Let P be a NAND-CIRC program of n inputs, m outputs, and s lines, and let t be the number of distinct variables used by P. The **list of tuples representation of** P is the triple (n, m, L), where L is the list of triples of the form (i, j, k) for $i, j, k \in [t]$. We assign a number for a variable of P as follows:

1. For every $i \in [n]$, the variable `X[i]` is assigned to the number i.
2. For every $j \in [m]$, the variable `Y[j]` is assigned to the number $t - m + j$.
3. Every other variable is assigned a number in $\{n, n+1, ..., t-m-1\}$ in the order in which the variable appears in the program P.

This is usually the default representation for NAND-CIRC programs, so we will call it "the representation" shorthand. The program could be represented as the list L instead of the triple (n, m, L).

Example 2.6 (Representing the XOR Program as a String)

To represent the XOR program of lines

```
u = NAND(X[0], X[1])
v = NAND(X[0], u)
w = NAND(X[1], u)
Y[0] = NAND(v, w)
```

we represent it as the tuple

$$L = \big((2,0,1),(3,0,2),(4,1,2),(5,3,4)\big) \tag{45}$$

Note that the variables `X[0], X[1]` are given the indices $0, 1$, the variable `Y[0]` is given the index 5, and the variables `u, v, w` are given the indices $2, 3, 4$.

So, if P is a program of size s, then the number t of variables is at most $3s$. Therefore, we can encode every variable index in $[t]$ as a string of length $l = \lceil \log(3s) \rceil$ (in binary), by adding leading zeros as needed. Since this is fixed-length encoding, it is prefix free, and so we can encode the list L of s triples as simply as the string of length $3ls$ obtained by concatenating all of these encodings.

Letting $S(s)$ be the length of the string representing the list L corresponding to a size s program, we get

$$S(s) = 3sl = 3s \lceil \log(3s) \rceil \tag{46}$$

2.10 NAND-CIRC Interpreter in NAND-CIRC

Since we can represent programs as strings, we can also think of a program as an input to a function. In particular, for every natural number $s, n, m > 0$, we define the function

$$EVAL_{s,n,m} : \{0,1\}^{S(s)+n} \longrightarrow \{0,1\}^m \tag{47}$$

as such: Given that px is the concatenation of two strings $p \in \{0,1\}^{S(s)}$ representing a list of triples L that represents a size-s NAND-CIRC program P, and $x \in \{0,1\}^n$ is a string,

$$EVAL_{s,n,m}(px) = P(x) \tag{48}$$

where $P(x)$ is equal to the evaluation $P(x)$ of the program P on input x. If p is not the list of tuples representation of a NAND-CIRC program, then $EVAL_{s,n,m} = 0^m$ (error message). Some important properties of EVAL include:

1. $EVAL_{s,n,m}$ is a finite function takin a string of fixed length as input and outputting a string of fixed length as output.
2. $EVAL_{s,n,m}$ is a single function, such that computing $EVAL_{s,n,m}$ allows us to evaluate *arbitrary* NAND-CIRC programs of a certain lenfth on *arbitrary* inputs of the appropriate length.
3. $EVAL_{s,n,m}$ is a *function*, not a *program*. That is, $EVAL_{s,n,m}$ is a *specification* of what output is associated with what input. The existence of a *program* that computes $EVAL_{s,n,m}$ (i.e. an *implementation* for $EVAL_{s,n,m}$) is a separate fact, which needs to be established.

Theorem 2.13 (Existence of a Bounded Universal Program)

For every $s, n, m \in \mathbb{N}$ with $s \geq m$, there is a NAND-CIRC program $U_{s,n,m}$ that computes the function $EVAL_{s,n,m}$.

That is, the NAND-CIRC program $U_{s,n,m}$ takes the description of *any other NAND-CIRC program* P (of the right length and inputs/outputs) and *any input* x, and computes the result of evaluating the program

P on the input x. Given the equivalence between NAND-CIRC programs and Boolean circuits, we can also think of $U_{s,n,m}$ as a circuit that takes as inputs the description of other circuits and their inputs, and returns their evaluation.

Definition 2.26 (Bounded Universal Program)

We call this NAND-CIRC program $U_{s,n,m}$ that computes $EVAL_{s,n,m}$ a **bounded universal program**, or a **universal circuit**. It is "universal" in the sense that this is a *single program* that can evaluate arbitrary code, where "bounded" stands for the fact that $U_{s,n,m}$ only evaluates programs of bounded size.

This theorem is profound because it proves the existence of a NAND-CIRC program that takes in *another* NAND-CIRC program along with its input. But it provides no explicit bound on the size of this program. The following theorem takes care of that.

Theorem 2.14 (Efficient bounded universality of NAND-CIRC programs)

For every $s, n, m \in \mathbb{N}$, there is a NAND-CIRC program of at most $O(s^2 \log s)$ lines that computes the function

$$EVAL_{s,n,m} : \{0,1\}^{S+n} \longrightarrow \{0,1\}^m \tag{49}$$

defined above (where S is the number of bits needed to represent programs of s lines). This allows us to place an upper bound on the size of $U_{s,n,m}$ that is *polynomial* in its input length.

3 Regular Languages

We now extend our definition of computational tasks to consider functions with the *unbounded* domain. The big takeaway from this chapter is that we can think of an algorithm as a "finite answer to an infinite number of questions." To express an algorithm, we need to write down a finite set of instructions that will enable us to compute on arbitrarily long inputs.

Example 3.1 (XOR as a Parity Function)

Note that the function $XOR : \{0,1\}^* \longrightarrow \{0,1\}$ equals 1 iff the number of 1's in x is odd. At best, we can compute XOR_n, the restriction of XOR to $\{0,1\}^n$ with NAND-CIRC programs.

Example 3.2 (The Multiplication Function)

The multiplication function takes the binary representation of a pair of integers $x, y \in \mathbb{N}$ and outputs the binary representation of the product $x \cdot y$.

$$MULT : \{0,1\}^* \times \{0,1\}^* \longrightarrow \{0,1\}^* \tag{50}$$

Since we can represent a pair of strings as a single string, we will consider functions such as MULT as

$$MULT : \{0,1\}^* \longrightarrow \{0,1\}^* \tag{51}$$

Example 3.3 (Palindrome function)

Another example of an infinite function is

$$PALINDROME(x) = \begin{cases} 1 & \forall i \in \|x\|,\ x_i = x_{|x|-i} \\ 0 & else \end{cases} \tag{52}$$

which outputs 1 if x is a (base-2) palindrome and 0 if not.

Definition 3.1 (Booleanizing a Function)

Sometimes, we can obtain a Boolean variant of a non-Boolean function. This process is called **booleanizing**.

Example 3.4 (Boolean variant of MULT)

The following is a boolean variant of MULT

$$BMULT(x, y, i) = \begin{cases} i\text{th bit of } x \cdot y & i < |x \cdot y| \\ 0 & else \end{cases} \tag{53}$$

Note that if we can compute $BMULT$, we can compute MULT as well, and vice versa.

3.1 Regular Grammars

Definition 3.2 (Regular Grammar)

A grammar G is **regular** if it is either of the following:

1. *Right Linear.*
$$R \subset V \times (\Sigma \cup \Sigma V \cup \{e\}) \tag{54}$$
Meaning that every rule must look like $A \to a$ or $A \to aB$.
2. *Left Linear.*
$$R \subset V \times (\Sigma \cup V\Sigma \cup \{\epsilon\}) \tag{55}$$
Meaning that every rule must look like $A \to a$ or $A \to Ba$.

3.2 Regular Expressions

Searching for a piece of text is a common task in computing. At its heart, the *search problem* is quite simple. We have a collection $X = \{x_0, ..., x_k\}$ of strings (e.g. files on a hard-drive, or student records in a database), and the user wants to find out the subset of all the $x \in X$ that are *matched* by some pattern. In full generality, we can allow the user to specify the pattern by specifying a (computable) function $F : \{0,1\}^* \longrightarrow \{0,1\}$, where $F(x) = 1$ corresponds to the pattern matching x. That is, the user provides a program P and the system returns all $x \in X$ such that $P(x) = 1$.

However, we don't want our system to get into an infinite loop just trying to evaluate the program P. For this reason, typical systems for searching files or databases do not allow users to specify the patterns using full-fledged programming languages. Rather, such systems use restricted computational models that on the one hand are rich enough to capture many of the queries needed in practice, but on the other hand are restricted enough so that queries can be evaluated very efficiently on huge files and in particular cannot result in an infinite loop. One of the most popular such computational models is *regular expressions*.

Definition 3.3 (Regular Expression)

Given an alphabet Σ, a **regular expression (regex)** e is defined with the following axioms.

1. *Symbol.* Any character $a \in \Sigma$ is a regex.
2. *Epsilon.* ϵ, which stands for the empty string, is a regex.
3. *Or.* Given regexes R_1, R_2, $R_1 \mid R_2$, defined to be either R_1 or R_2, is a regex.
4. *Concatenation.* Given regexes R_1, R_2, R_1R_2, defined to be the concatenation of them, is a regex.
5. *Kleene Star.* Given regex R, R^* represents a concatenation of 0 or more R.[a]

[a] Note that this is essential since the number of concatenations is unbounded. It is *not* syntactic sugar.

Example 3.5 (Regular Expressions over Binary Alphabet)

The following are regular expressions over the alphabet $\{0,1\}$.
$$\big(00(0^*)|11(1^*)\big)^*, \qquad 00^*|11 \tag{56}$$

To make things more convenient to write, we use the following common *syntactic sugar*. This doesn't add functionality.

1. *1 or More.* $R^+ = RR^*$
2. *Optional.* $R? = R \mid \epsilon$[3]

[3] This is pretty much the only use case for ϵ, so you really never write down ϵ directly in a regex.

3. *Any Character.* . stands for any character.
4. *Brackets.* This just means a big "or" of all the characters. We can write $[abc] = (a|b|c)$, and can do $[0-9]$, $[a-z]$.
5. *Negation.* `[R̂]` means any character that is not R.

We can have more syntactic sugar, but these are the most common. Now, we can formally define what it means for a string to match a RegEx.

Definition 3.4 (Matching a RegEx)

Let e be a regular expression over the alphabet Σ. The function $\Phi_e : \Sigma^* \longrightarrow \{0, 1\}$ is defined as follows:

1. If $e = \sigma$, then $\Phi_e(x) = 1$ iff $x = \sigma$
2. If $e = (e' \mid e'')$, then $\Phi_e(x) = \Phi_{e'}(x) \vee \Phi_{e''}(x)$ where $\vee$ is the OR operator.
3. If $e = (e')(e'')$, then $\Phi_e(x) = 1$ iff there is some $x', x'' \in \Sigma^*$ such that x is the concatenation of x' and x'' and $\Phi_{e'}(x') = \Phi^{e''}(x'') = 1$
4. If $e = (e')^*$ then $\Phi_e(x) = 1$ iff there is some $k \in \mathbb{N}$ and some $x_0, x_1, ..., x_{k-1} \in \Sigma^*$ such that x is the concatenation $x_0, x_1, ..., x_{k-1}$ and $\Phi_{e'}(x_i) = 1$ for every $i \in [k]$.
5. For the edge cases, $\Phi_\emptyset$ is the 0 function, and $\Phi_{\text{""}}$ is the function that only outputs 1 on the empty string "".

It is said that a regular expression e over Σ **matches** a string $w \in \Sigma^*$—or equivalently, e **accepts** string w—if $\Phi_e(x) = 1$. The process of matching a given regex to an arbitrary string is called **computing the regex**.

Example 3.6 (Matching a RegEx)

Let us have a string `abcdef` and regex

```
a (b | g) e? cd (ef)*
```

Does this string match the regex? Yes.

Definition 3.5 (Recognized Language of a RegEx)

The set of all strings $w \in \Sigma^*$ that matches a regex R is called the **language generated by** R.

Example 3.7 (Generated Language of all C Identifiers)

The syntax for all numbers might look something like

```
0 | (-? [1-9][0-9])
```

Example 3.8 (Numeric Identifiers)

1. We may write `[1-9][0-9]*`, but this does not account for negative numbers.
2. We may write `-?[1-9][0-9]*`, but this does not account for 0.
3. We may write `0 | (-?[1-9][0-9]*)`. But should we include `-0`? This then becomes more of a design choice.

Every regular expression e corresponds to a function $\Phi_e : \Sigma^* \longrightarrow \{0,1\}$ where $\Phi_e(x) = 1$ if x *matches* the regular expression. The definition is tedious.

Definition 3.6 (Regular Boolean Function)

Given alphabet Σ and boolean function $F : \Sigma^* \longrightarrow \{0,1\}$, we say that F is **regular** if $F = \Phi_e$ for some regular expression e.

One of the nice things about regexs is that they aren't as powerful as arbitrary programming, which is nice since it limits the complexity. We can write a simple declarative specification, but we can't do things like the following. In fact, it is proven with math.

1. write balanced parentheses, i.e. write regexes that outputs all parentheses that are properly nested
2. type checking

So, with this in mind, we can write regex's for things that we might care about in programming. For keywords, it's trivial.

Example 3.9 (Counterexamples to Writing Balanced Parentheses)

3.3 Finite Automata

In general, computing regular expressions (i.e. matching a string with a regular expression) is hard. To see if we can make this easier, we can look at *DFAs*, which are generally seen as easy to compute.

Definition 3.7 (Deterministic Finite Automaton)

A **deterministic finite automaton (DFA)** is a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$ consisting of
1. a finite set of states Q
2. a finite set of input symbols called the alphabet Σ
3. a transition function $\delta : Q \times \Sigma \to Q$
4. an initial state $q_0 \in Q$
5. a set of accepting states $F \subset Q$

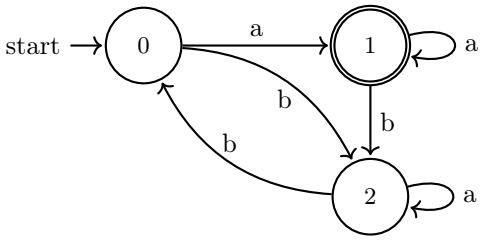


Figure 11: Note that by definition of the transition function, every single state must have exactly one outgoing transition for every symbol in Σ.

Definition 3.8 (DFA Computability)

We say that a boolean function $F : \{0,1\}^* \longrightarrow \{0,1\}$ is **DFA computable** if there exists some DFA that computes F.

Definition 3.9 (Recognized Language of a DFA)

Let $w = a_1 a_2 \dots a_n$ be a string over alphabet Σ. The DFA M **accepts** (or computes, or matches) the string w if a sequence of states $r_0, r_1, \dots, r_n$ exists in Q with the following conditions.

1. $r_0 = q_0$.
2. $r_{i+1} = \delta(r_i, a_{i+1})$ for $i = 0, \dots, n-1$
3. $r_n \in F$.

The set of all words that are accepted by a DFA $L(M)$ is called the **language generated by** M.

Definition 3.10 (Nondeterministic Finite Automaton)

A **nondeterministic finite automaton (NFA)** is a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$ consisting of

1. a finite set of states Q
2. a finite set of input symbols called the alphabet Σ
3. a transition function $\delta : Q \times (\Sigma \cup \{\epsilon\}) \to 2^{Q\,a}$
4. an initial state $q_0 \in Q$
5. a set of accepting states $F \subset Q$

where ϵ denotes an empty string. There are edges labeled with ϵ, which allows you to traverse to another node at no cost.

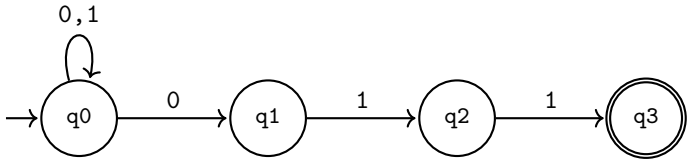


Figure 12: An NFA accepting strings that end in `011`. Nondeterminism is evident at state `q0` upon reading a `0`.

The **epsilon closure** of a set of nodes is the set plus any other nodes that you can traverse through ϵ-edges.

[a] 2^Q denotes the power set of Q.

Definition 3.11 (Recognized Language of an NFA)

Let $w = a_1 a_2 \dots a_n$ be a string over alphabet Σ. The DFA M **accepts** (or computes, or matches) the string w if a sequence of states $r_0, r_1, \dots, r_n$ exists in Q with the following conditions.

1. $r_0 = q_0$.
2. $r_{i+1} \in \delta(r_i, a_{i+1})$ for $i = 0, \dots, n-1$
3. $r_n \in F$.

The set of all words that are accepted by a DFA $L(M)$ is called the **language generated by** M.

Algorithm 3.1 (Converting DFA into NFA)

This may seem obvious, but we need to be slightly careful.

Definition 3.12 (Operations on Finite Automata)

Let M_1, M_2 be two finite automata over the same alphabet Σ. Then,

1. *Union.* The union $M_1 \cup M_2$ is defined to be the FA M satisfying $L(M) = L(M_1) \cup L(M_2)$.
2. *Complement.* The complement M_1^c is defined to be the FA M satisfying $L(M) = L(M_1)^c \subset \Sigma^*$.
3. *Intersection.* The intersection $M_1 \cap M_2$ is defined to be the DFA M satisfying $L(M) = L(M_1) \cap L(M_2)$.

Lemma 3.1 (Union of NFA)

The union of two NFAs M_1, M_2 is simple since you can make a new start node and connect it to the start nodes of the two NFAs with an ϵ-edge.

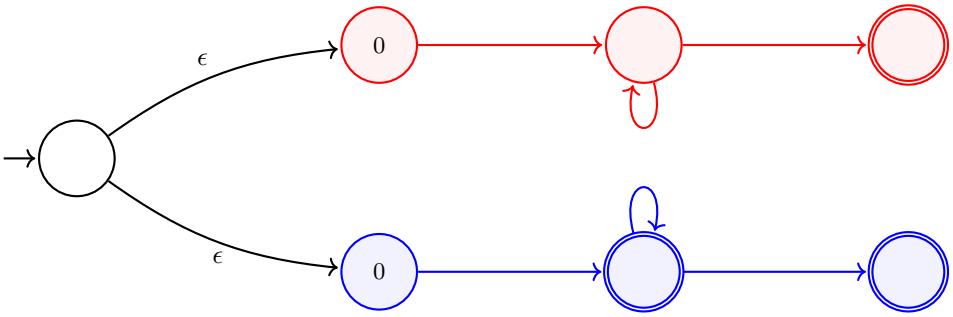


Figure 13: Combined NFA with epsilon transitions and a shared start node.

Lemma 3.2 (Union of DFA)

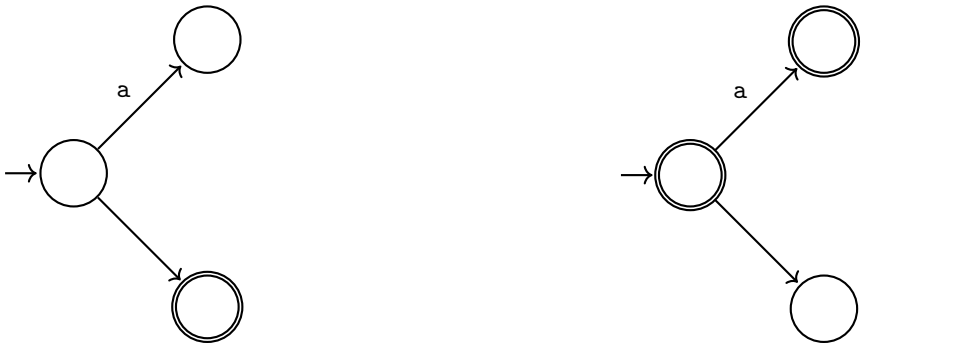
You should convert them to NFAs (which is trivial), then take the union, and then convert them back.

Lemma 3.3 (Complement of DFA)

We just flip all the accept and not accept states.

However, this doesn't work for an NFA.

Example 3.10 (Cannot Flip Accept States in NFA)



(a) Accepts `a`.

(b) Also accepts `a`, as well as the empty string.

Figure 14

Lemma 3.4 (Complement of NFA)

Therefore, you must turn an NFA into a DFA and do the complement, and then turn it back.

Theorem 3.5 (Accepting Nothing)

We can check if a DFA or NFA accepts nothing by doing BFS and seeing if we can get to an accepting state.

Theorem 3.6 (Equivalence of NFA/DFAs)

We can check if two DFAs or two NFAs are equivalent using the rule

$$A = B \iff \overline{A} \cap B\emptyset, A \cap \overline{B} = \emptyset \tag{57}$$

Therefore, we can check if two regex's are equivalent by converting them into DFAs first.

Note that checking equivalence is a very nontrivial thing for algorithms, and in fact is provably impossible to check for arbitrary algorithms (called undecidability).

In general, DFAs are known to be the easiest to compute. You can implement this basically as a giant hash table, with the accepting states stored in a list or something.

Example 3.11 (Computing a DFA)

Consider the DFA above, and the string `ababba`.

In general, DFAs are preferred by computers, while humans prefer regular expressions. We want to bridge them somehow so that we can convert regexes to DFAs, and we do this with nondeterministic finite automata. This allows us to write a nice declarative.

The problem with NFAs is that it may take an exponential time to compute due to possible branching factors at every node. It turns out that we can turn an NFA into a DFA, which may theoretically have exponentially

more nodes, but in general does not.

Algorithm 3.2 (Converting NFA to DFA)

This is basically a fancy BFS algorithm. A DFA state is going to be a set of NFA states. Finally, the accepting state of the NFA is any state that contains the accepting state of the DFA.

Example 3.12 (Converting NFA to DFA)

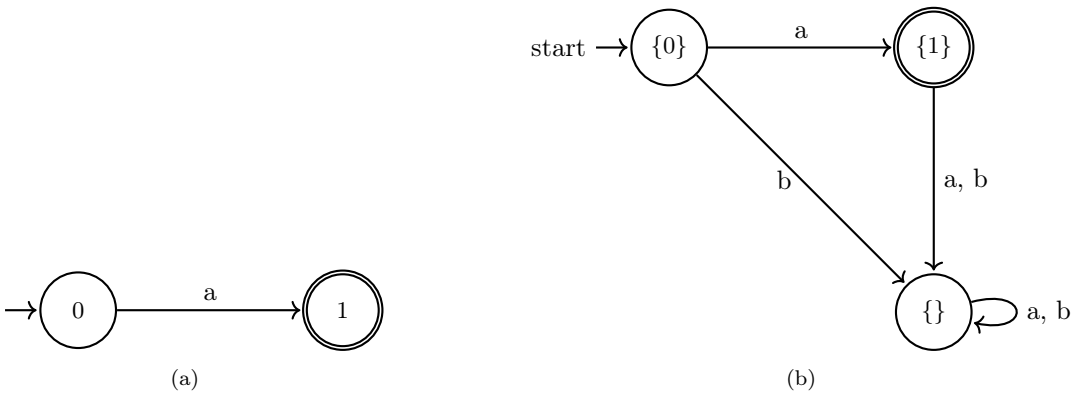


Figure 15

Example 3.13 (Converting NFA to DFA)

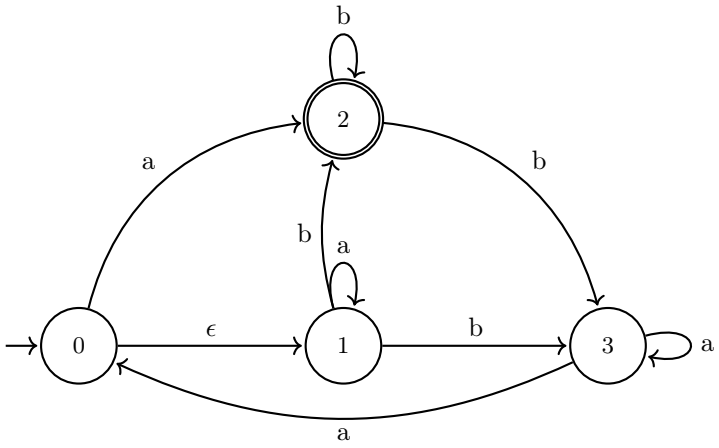


Figure 16: NFA.

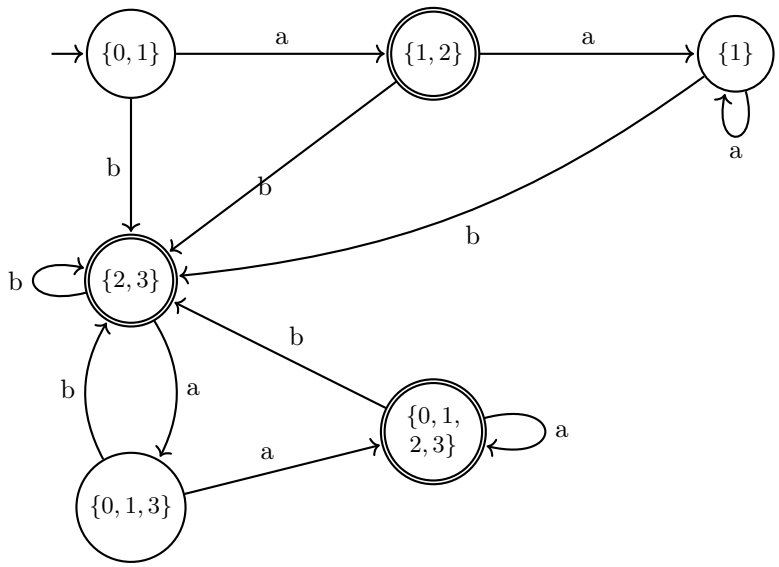


Figure 17: DFA.

Algorithm 3.3 (Converting RegEx to NFA)

This is a recursive algorithm.

1. *Symbol.* A symbol $a \in A$ is converted to a simple one edge graph. This is one of the base cases.

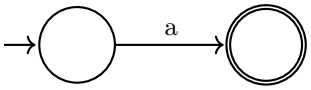


Figure 18

2. *Epsilon.* The ϵ is converted to just an accepting state. Note that it can't take in an input. This is another base case.



Figure 19

3. *Or.* Given two NFAs representing R_1 and R_2, we take the start node and connect it to the start nodes of the two NFAs with an ϵ-edge.[a]

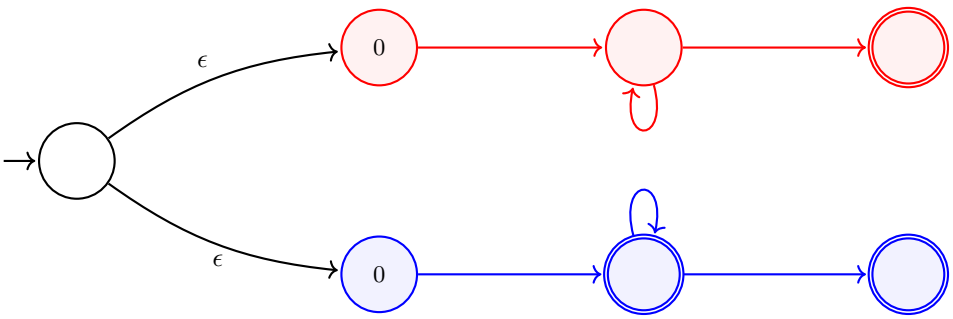


Figure 20: Combined NFA with epsilon transitions and a shared start node.

4. *Concatenation.* For R_1R_2, we combine the two NFAs by taking all accepting states in R_1 and connect them to the start of R_2 with an ϵ-edge. Then, we change the accepting states of R_1 to regular states.

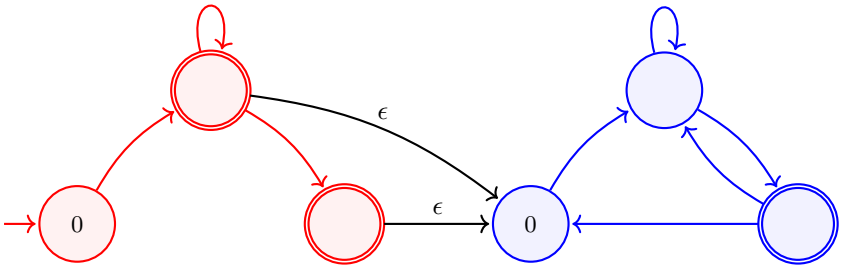


Figure 21: Sequential composition of two NFAs via epsilon transition.

5. *Kleene Start.* Given the NFA of a regex R with start node 0, to find the NFA of R^*, we do the following. First, make a new start accepting node s and draw an edge $s \xrightarrow{\epsilon} 0$. This makes 0 not a start node anymore. Finally, for each accepting node a in the NFA of R, draw edges $a \xrightarrow{\epsilon} s$.

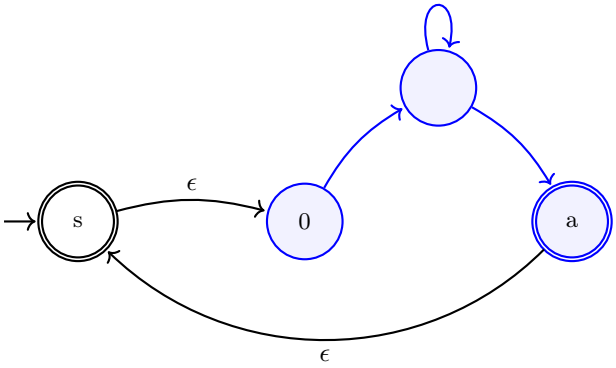


Figure 22: The new start node is necessary to avoid the possibility of going around in the NFA and ending at the old start node.

[a] Note that the presence of ϵ-edges makes things a lot easier for us.

So given a regular expression, we convert this to an NFA and then a DFA, which may be (but generally not) exponential complexity with respect to the regex alphabet. But it is independent of the length of the string that we are matching. The matching itself is linear, so we can run millions of bytes-long strings through this

DFA.

So far, establishing these algorithms to compute DFAs doesn't have an obvious connection to NFAs. It turns out that lexers end up having a bunch of DFAs in them, with different accepting states for different tokens.

Example 3.14 (NFAs are More Verifiable to Humans)

Consider the language L of all comments that have delimiters of the form `/# ... #/`. How would we write this as a regular expression?

1. Our first intuition would be something like `/#(.*)#/`, but this includes strings of form `/##//##/`, which is two comments.
2. We may try to write `/#([^#]|#[^/])*#/`, but this includes strings of form `/###/...#/`, which again isn't viable.

Generally, when you have to keep thinking of edge cases and you're hacking things, you're on the losing side. However, if we think of this in terms of an NFA, this becomes much better.

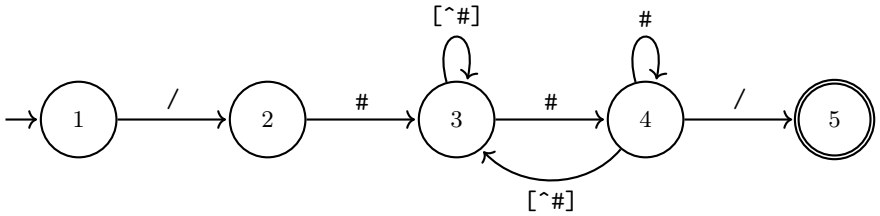
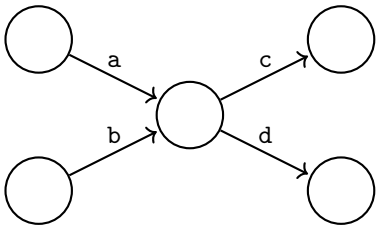


Figure 23: An NFA is much more intuitive to verify.

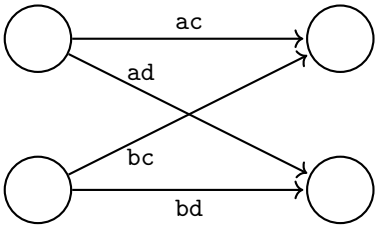
Algorithm 3.4 (NFA to RegEx)

There are three rules to converting an NFA M with start state s and accepting states F to RegEx's. To do this, we actually put this into an intermediate form called a *generalized NFA*, which has regex's in edge labels, not just symbols. First, add a new start S and connect $S \xrightarrow{\epsilon} s$. Also add a new "end state" E and for every $f \in F$, connect $f \xrightarrow{\epsilon} E$. We do this is so that we avoid messy problems where there are multiple edges or loops.

1. *Eliminate State without Loop.* To delete a node that doesn't have an outgoing edge that loops back to itself, look at all incoming edges and outgoing edges, and directly connect all edges.



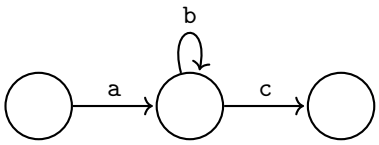
(a) NFA with intermediate state.



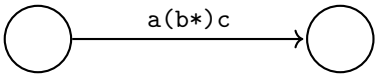
(b) Direct connections after state removal.

Figure 24: Comparison of an NFA before and after removing a central state via state elimination.

2. *Eliminate State with Loop.* To delete a node that does have an outgoing edge that loops back to itself,



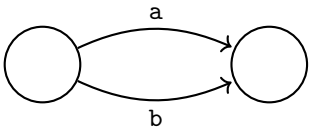
(a) NFA with a self-looping intermediate state.



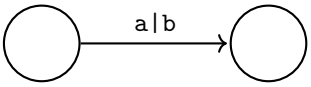
(b) Resulting NFA after eliminating the middle state.

Figure 25: Demonstration of the state elimination rule for self-loops.

3. *Collapse Multiple Edges Between 2 States.* If there are two nodes n, m with multiple edges pointing from n to m, then we can just replace it with a single edge that represents an or.



(a) Two nodes connected by parallel edges.



(b) Parallel edges combined using the union operator.

Figure 26: State elimination rule for combining parallel edges into a single regular expression.

At the end, we will have one edge from s

Example 3.15 (Deriving RegEx for Comments from NFA)

Let's do the conversion on the NFA above.

1. *Start.* We add the new start and end states.

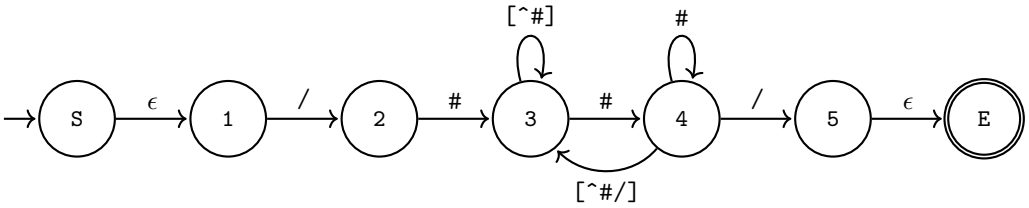


Figure 27: Condensed NFA with `node distance=2cm`.

2. *State 2.*

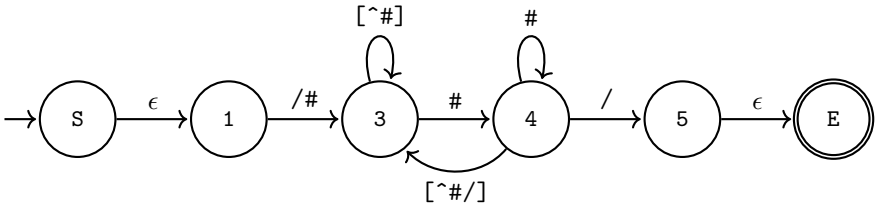


Figure 28: Reduced NFA with `node distance=2cm` and corrected `^` notation.

3. *State 1.*

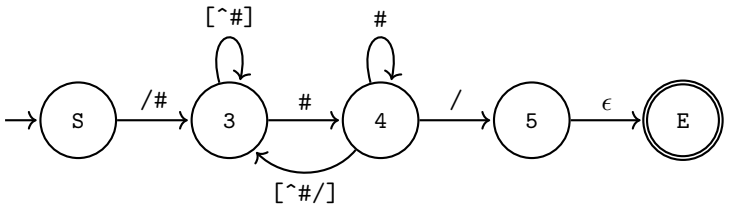


Figure 29: The NFA after eliminating state `1`. The transition label `/#` is moved to the edge starting from `S`.

4. *State 5.*

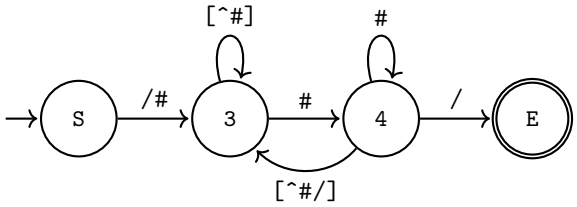


Figure 30: The NFA after eliminating state `5`. The transition to the end state `E` is now the single character `/`.

5. *State 4.* Since state 4 has a loop, we use the second rule. It has an incoming edge from 3 with a `#`, a loop edge with a `#`, and an outgoing edge to state E with a `/` and another outgoing edge to state 3 with a `[^#]`. Therefore,

$$(3 \xrightarrow{\texttt{\#}} 4 \xrightarrow{\texttt{/}} E) \Rightarrow (3 \xrightarrow{\texttt{\#(\#*)/}} E) = (3 \xrightarrow{\texttt{(\#+)/}} E) \tag{58}$$

$$(3 \xrightarrow{\texttt{\#}} 4 \xrightarrow{\texttt{[\^\#]}} 3) \Rightarrow (3 \xrightarrow{\texttt{\#(\#*)[\^\#]}} 3) = (3 \xrightarrow{\texttt{(\#+)[\^\#]}} 3) \tag{59}$$

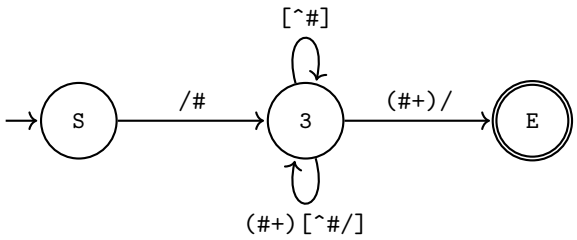


Figure 31: The NFA after eliminating state `4`. The paths through state 4 are converted into a direct edge to `E` and an additional loop component on state `3`.

6. *State 3.* Since there are two edges that have the same source and destination nodes (both state 3), we should collapse them using rule 3.

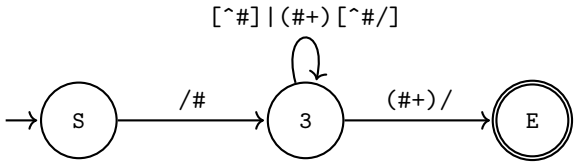


Figure 32: The NFA after eliminating state `4`. The paths through state 4 are converted into a direct edge to `E` and an additional loop component on state `3`.

7. *State 3.* Now we can get rid of state 3.

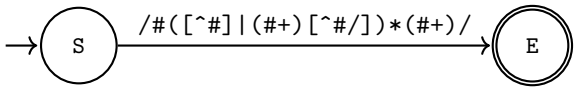


Figure 33: The final NFA after eliminating all intermediate states. The label on the edge represents the final regular expression.

Now we are done. Our regex is

```
/\#([\^{}\#]|(\#+)[\^{}\#/])*(\#+)/
```

3.4 Equivalent Forms of Regularity

Theorem 3.7 (Equivalence of Regular Language)

Let L be a formal language over alphabet Σ. The following are equivalent.
1. L is generated by a regular grammar G.
2. L is generated by a DFA.
3. L is generated by regular expression R.

Proof.

4 Context Free Languages

4.1 Context Free Grammars

When a person designs a programming language, they need to determine its *syntax*. That is, the designer decides which strings correspond to valid programs, and which ones do not (i.e. which strings contain a syntax error). To ensure that a compiler or interpreter always halts when checking for syntax errors, language designers typically *do not* use a general Turing-complete mechanism to express their syntax. Rather, they use a *restricted* computational model, most often being *context free grammars*.

Example 4.1 (Arithmetic Function)

Consider the function $ARITH : \Sigma^* \longrightarrow \{0, 1\}$ that takes as input a string x over alphabet

$$\Sigma = \{(,), +, -, \times, \div, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\} \tag{60}$$

and returns 1 if and only if the string x represents a valid arithmetic expression. Intuitively, we build expressions by applying an operation such as $+, -, \times, \div$ to smaller expressions or enclosing them in parentheses. More precisely, we can make the following definitions:

1. A *digit* is one of the symbols $0, 1, 2, 3, 4, 5, 6, 7, 8, 9$.
2. A *number* is a sequence of digits (we will drop the condition that the sequence does not have a leading zero)
3. An *operation* is one of $+, -, \times, \div$.
4. An *expression* has either the form
   (a) *"number"*
   (b) *"sub-expression1 operation sub-expression2*
   (c) *"(sub-expression1)"*
   where "sub-expression1" and "sub-expression2" are themselves expressions. Note that this is a recursive function.

A context free grammar (CFG) is a formal way of specifying such conditions, consisting of a set of rules that tell us how to generate strings from smaller components.

Definition 4.1 (Context Free)

A grammar is **context free**[a] if

$$R \subset V \times (V \cup \Sigma)^* \tag{61}$$

That is, it asserts that the left-hand side can only contain a single non-terminal. A language L is **context-free** if $L = L(G)$ for some context-free grammar G.

[a] For theorists, context sensitive grammars are important, but for engineers, context free grammars matter much more.

Example 4.2 (A Grammar for Arithmetic Expressions)

The example of well-formed arithmetic expressions can be captured formally by the following context free grammar.

1. The alphabet Σ is $\{(,), +, -, \times, \div, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$.
2. The variables are $V = \{expression, number, digit, operation\}$
3. The rules are the set R containing the following 19 rules:
   (a) 4 Rules: $operation \implies +$, $operation \implies -$, $operation \implies \times$, $operation \implies \div$
   (b) 10 Rules: $digit \implies 0$, $digit \implies 1$, ..., $digit \implies 9$
   (c) Rule: $number \implies digit$

(d) Rule: $number \implies digitnumber$
(e) Rule: $expression \implies number$
(f) Rule: $expression \implies expression\ operation\ expression$
(g) Rule: $expression \implies (expression)$
4. The starting variable is $expression$.

This allows for a parse-tree structure because each variable "branches" into a new string independently of its neighbors.

Definition 4.2 (Ambiguous)

A context-free grammar is **ambiguous** if there is a string that can have more than one valid derivation tree.

Example 4.3 (Ambiguous Context Free Grammar)

Let $V = \{E\}$ be the non-terminals, and $\{+, -, \texttt{num}\}$ be our terminals, which come from our lexer. Then, our production rules can look something like.
1. $E \to E + E$.
2. $E \to E - E$.
3. $E \to \texttt{num}$.

So basically, if we have some word, then we can replace any E in the word by any of the three choices on the right hand side. For example, we can do the following sequence of derivations.

$$\begin{aligned} E &\Rightarrow E + E && (62)\\ &\Rightarrow E - E + E && (63)\\ &\Rightarrow E - \texttt{num} + E && (64)\\ &\Rightarrow E + E - \texttt{num} + E && (65) \end{aligned}$$

You can keep doing this until there is no more production rules you can apply. For context free grammars, you basically do this until there are only terminals left, e.g. $\texttt{num} + \texttt{num} - \texttt{num} + \texttt{num}$. But note that this is ambiguous since there are multiple ways to derive. For example, if we wanted to get something like `num - num - num`, we can do it in either of the following ways.

$$\begin{aligned} E &\Rightarrow E - E && (66)\\ &\Rightarrow E - (E - E) && (67)\\ &\Rightarrow \texttt{num} - (\texttt{num} - \texttt{num}) && (68)\\ E &\Rightarrow E - E && (69)\\ &\Rightarrow (E - E) - E && (70)\\ &\Rightarrow (\texttt{num} - \texttt{num}) - \texttt{num} && (71) \end{aligned}$$

This is not good, so we want to modify our grammar so that it is safe from these ambiguities.

Example 4.4 (Non-Ambiguous Context-Free Grammar)

If we have a grammar with the following production rules.
1. $E \to E + \texttt{num}$
2. $E \to E - \texttt{num}$
3. $E \to \texttt{num}$

Then, we have no ambiguities since there is only one possible way

$$\begin{aligned} E &\Rightarrow E - \texttt{num} && (72)\\ &\Rightarrow (E - \texttt{num}) - \texttt{num} && (73)\\ &\Rightarrow (\texttt{num} - \texttt{num}) - \texttt{num} && (74) \end{aligned}$$

Note that figuring out whether a grammar is ambiguous or unambiguous is a bit tricky. It is in fact undecidable.

5 Context Sensitive Languages

Definition 5.1 (Context Sensitive)

A grammar is **context sensitive** if

$$R \subset \{(\alpha, \beta) \mid \alpha, \beta \in (V \cup \Sigma)^+, |\alpha| \leq |\beta|\} \cup \{(S, \epsilon) \text{ if S does not appear on any RHS}\} \tag{75}$$

That is, the length of the string on the right must be greater than or equal to the length of the string on the left. You cannot "delete" symbols are you derive. A language L is **context-sensitive** if $L = L(G)$ for some context-sensitive grammar G.

6 Turing Machines

Similar to how a person does calculations by reading from and writing to a single cell of a paper at a time, a Turing machine is a hypothetical machine that reads from its "work tape" a single symbol from a finite alphabet Σ and uses that to update its state, write to tape, and possibly move to an adjacent cell. To compute a function F using this machine, we initialize the tape with the input $x \in \{0,1\}^*$ and our goal is to ensure that the tape will contain the value $F(x)$ at the end of the computation. Specifically, a computation of a Turing machine M with k states and alphabet Σ on input $x \in \{0,1\}^*$ is formally defined as follows.

Definition 6.1 (Turing Machine)

A (one tape) **Turing machine** with k states and alphabet $\Sigma \supset \{0,1,\triangleright,\emptyset\}$ is represented by a **transition function**

$$\delta_M : [k] \times \Sigma \longrightarrow [k] \times \Sigma \times \{\mathsf{L},\mathsf{R},\mathsf{S},\mathsf{H}\} \tag{76}$$

For every $x \in \{0,1\}^*$, the *output* of M on input x, denoted by $M(x)$, is the result of the following process:

1. We initialize T to be the infinite sequence (also represented by a tape)

$$\triangleright, x_0, x_1, ..., x_{n-1}, \emptyset, \emptyset, ... \tag{77}$$

   where $n = |x|$. That is, $T[0] = \triangleright, T[i+1] = x_i$ for $i \in [n]$, and $T[i] = \emptyset$ for $i > n$.)
2. We also initialize $i = 0$ (the head is at the starting position) and we begin with the initial state $s = 0$, $s \in [k]$.
3. We then repeat the following process which is defined according to the transition function:
   (a) Let $(s', \sigma', D) = \delta_M(s, T[i])$.
   (b) Set $s \to s'$, $T[i] \to \sigma'$
   (c) If $D = \mathsf{R}$, then set $i \to i+1$, if $D = \mathsf{L}$, then set $i \to \max\{i-1, 0\}$. If $D = \mathsf{S}$, then we keep i the same.
   (d) If $D = \mathsf{H}$, then halt.

   Colloquially, at each step, the machine reads the symbol $\sigma \in T[i]$ that is in the ith location of the tape. Bsaed on this symbol and its state s, the machine decides on
   (a) What symbol σ' to write on the tape
   (b) Whether to move Left ($i \to i-1$), Right ($i \to i+1$), Stay in place, or Halt the computation
   (c) What is going to be the new state $s \in [k]$
4. If the process above halts, then M's output, denoted by $M(x)$ is the string $y \in \{0,1\}^*$ obtained by concatenating all the symbols in $\{0,1\}$ in positions $T[0], ..., T[i]$ where $i+1$ is the first location in the tape containing $\emptyset$.
5. If the Turing machine does not halt then we denote $M(x) = \perp$.

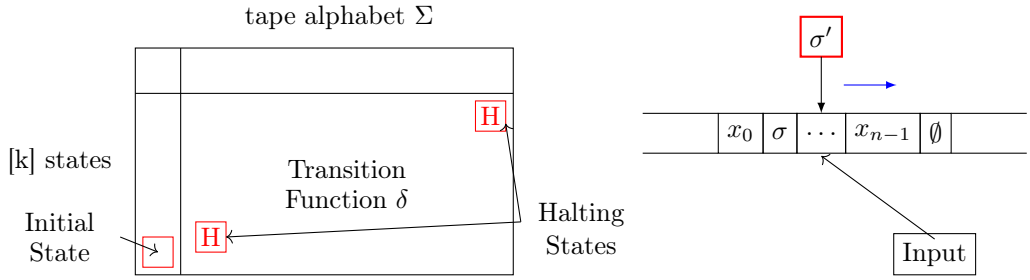


Figure 34: We can visualize a Turing machine as a table and a tape labeled below.

In fact, all modern computing devices are Turing machines at heart. You input a string of bits, the machine flips a bunch of switches, and outputs another string of bits.

Example 6.1 (Turning Machine for Palindromes)

Let PAL be the function that on input $x \in \{0,1\}^*$, outputs 1 if and only if x is an (even length) *palindrome*, in the sense that

$$x = w_0 ... w_{n-1} w_{n-1} w_{n-2} ... w_0 \tag{78}$$

for some $n \in \mathbb{N}$ and $w \in \{0,1\}^*$. We will now describe a Turing machine that computes PAL. To specify M, we need to specify

1. M's tape alphabet Σ which should contain at least the symbols 0, 1, $\triangleright$, and $\emptyset$, and
2. M's transition function which determines what action M takes when it reads a given symbol while it is in a particular state.

For this specific Turing machine, we will use the alphabet $\{0, 1, \triangleright, \emptyset, \times\}$ and will have $k = 13$ states, with the following labels for the numbers.

State	Label	State	Label
0	`START`	7	`ACCEPT`
1	`RIGHT_0`	8	`OUTPUT_0`
2	`RIGHT_1`	9	`OUTPUT_1`
3	`LOOK_FOR_0`	10	`0_AND_BLANK`
4	`LOOK_FOR_1`	11	`1_AND_BLANK`
5	`RETURN`	12	`BLANK_AND_STOP`
6	`REJECT`		

The operation of our Turning machine, in words, is as such:

1. M starts in the state `START` and goes right, looking for the first symbol that is 0 or 1. If it finds $\emptyset$ before it hits such a symbol then it moves to the `OUTPUT_1` state.
2. Once M finds such a symbol $b \in \{0,1\}$, M deletes b from the tape by writing the $\times$ symbol, it enters either the `RIGHT_0` or `RIGHT_1` mode according to the value of b and starts moving rightwards until it hits the first $\emptyset$ or $\times$ symbol.
3. Once M finds this symbol, it goes into the state `LOOK_FOR_0` or `LOOK_FOR_1` depending on whether it was in the state `RIGHT_0` or `RIGHT_1` and makes one left move.
4. In the state `LOOK_FOR_`b, M checks whether the value on the tape is b. If it is, then M deletes it by changing its value to $\times$, and moves to the state `RETURN`. Otherwise, it changes to the `OUTPUT_0` state.
5. The `RETURN` state means that M goes back to the beginning. Specifically, M moves leftward until it hits the first symbol that is not 0 or 1, in which case it changes its state to `START`.
6. The `OUTPUT_`b states mean that M will eventually output the value b. In both the `OUTPUT_0` and `OUTPUT_1` states, M goes left until it hits $\triangleright$. Once it doe sso, it makes a right step and changes to the `1_AND_BLANK` or `0_AND_BLANK` state respectively. In the latter states, M writes the corresponding value, moves right and changes to the `BLANK_AND_STOP` state, in which it writes $\emptyset$ to the tape and halts.

The above description can be turned into a table describing for each one of the $13 \cdot 5 = 65$ combinations of state and symbol, what the Turing machine will do when it is in that state and it reads that symbol. This table is the *transition function* of the Turing machine.

Definition 6.2 (Computable Functions)

Let $F : \{0,1\}^* \longrightarrow \{0,1\}^*$ be a (total) function and let M be a Turing machine. We say that M **computes** F if for every $x \in \{0,1\}^*$, $M(x) = F(x)$. We say that a function F is **computable** if there exists a Turing machines M that computes it.

It turns out that being computable in the sense of a Turing machine is equivalent to being computable in virtually any reasonable model of computation. This statement is known as the **Church-Turing Thesis**. Therefore, this definition allows us to precisely define what it means for a function to be computable by *any possible algorithm*.

Definition 6.3 (The class R)

We define **R** to be the set of all computable functions $F : \{0,1\}^* \longrightarrow \{0,1\}$.

6.1 NAND-TM Programs

In addition to having a physical interpretation, Turing machines can also be interpreted as programs.

1. The *tape* becomes a *list* or *array* that can hold values from the finite set Σ.
2. The *head position* can be thought of as an integer-valued variable that holds integers of unbounded size.
3. The *state* is a *local register* that can hold one of a fixed number of values in $[k]$.

In general, every Turing machine M is equivalent to a program similar to the following:

```
#Gets an array Tape initialized to [">", x_0,..., x_(n-1), " ", " ", ...]
def M(Tape):
    state = 0
    i     = 0  #holds head location
    while(True):
        #Move head, modify state, write to tape based on current state and
        #cell at head below are just examples for how program looks
        #for a particular transition function
        if Tape[i]=="0" and state==7: #T_M(7,"0")=(19,"1","R")
            i += 1
            Tape[i]="1"
            state = 19
        elif Tape[i]==">" and state == 13: #T_M(13,">")=(15,"0","S")
            Tape[i] ="0"
            state = 15
        elif...
            ...
        elif Tape[i]==">" and state == 29: #T_M(29,">")=(.,.,"H")
            break  #Halt
```

If we were using Boolean variables, then we can encode the `state` variables using $\lceil \log k \rceil$ bits.

Note that in the code above, two new concepts are introduced:

1. *Loops*: NAND-CIRC is a straight line programming language. That is, a NAND-CIRC program of s lines takes exactly s steps of computation and hence in particular, cannot even touch more than $3s$ variables. *Loops* allow us to use a fixed-length program to encode the instructions for a computation that can take an arbitrary amount of time.
2. *Arrays*: A NAND-CIRC program of s lines touches at most $3s$ variables. While we can use variables with names such as `Foo_17` or `Bar[22]` in NAND-CIRC, they are not true arrays, since the number in the identifier is a constant that is not "hardwired" into the program. NAND-TM contains actual arrays that can have a length that is not a priori bounded.

The following equation summarizes the concepts:

$$\text{NAND-TM} = \text{NAND-CIRC} + \text{loops} + \text{arrays} \tag{79}$$

Surprisingly, adding loops and arrays to NAND-CIRC is enough to capture the full power of all programming languages. Hence, we could replace NAND-TM with any of Python, C, Javascript, etc.

Concretely, the NAND-TM programming language adds the following features on top of NAND-CIRC:

1. We add a special *integer valued variable* i. All other variables in NAND-TM are Boolean valued (as in NAND-CIRC).
2. Apart from i, NAND-TM has two kinds of varibales: *scalars* and *arrays*. *Scalar* variables hold one bit (just as in NAND-CIRC). *Array* variables hold an unbounded number of bits. At any point in the computation we can access the array variables at the location indexed by `i` using `Foo[i]`. We cannot access the arrays at loctions other than the one pointed by `i`.
3. We use the convention that *arrays* always start with a capital letter, and *scalar variables* (which are never indexed with `i`) start with lowercase letters. Hence, `Foo` is an array and `foo` is a scalar variable.
4. The input and output `X` and `Y` are not considered *arrays* with values of 0s and 1s.
5. We add a special `MODANDJUMP` instruction that takes two Boolean variables a, b as input and does the following:
   (a) If $a = 1, b = 1$, then `MODANDJUMP(a, b)` increments `i` by one and jumps to the first line of the program.
   (b) If $a = 0, b = 1$, then `MODANDJUMP`(a, b) decrements `i` by one and jumps to the first line of the program. If `i` already equals 0, then it stays at 0.
   (c) If $a = 1, b = 0$, then `MODANDJUMP`(a, b) jumps to the first line of the program without modifying `i`.
   (d) If $a = b = 0$, then `MODANDJUMP`(a, b) halts execution of the program.
6. The `MODANDJUMP` instruction always appears in the last line of a NAND-TM program and nowhere else.
7. Turing machines have the special symbol $\emptyset$ to indicate that tape location is "blank" or "uninitialized." In NAND-TM there is no such symbol, and all variables are *Boolean*, containing either 0 or 1. All variables and locations either default to 0 if they have not been initialized to another value. To keep track of whether a 0 in an array corresponds to a true 0 or to an uninitialized cell, a programmer can always add to an array `Foo` a *companion array* `Foo_nonblank` and set `Foo_nonblank[i]` to 1 whenever the `i`th location is initialized. In particular, we will use this convention for the input and output arrays `X` and `Y`. Therefore, a NAND-TM program has *four* special arrays `X, X_nonblank, Y, Y_nonblank`.

Therefore, when a NAND-TM program is executed on input $x \in \{0,1\}^*$ of length n, the first n cells of `X` are initialized to $x_0, ..., x_{n-1}$ and the first n cells of `X_noblank` are initalized to 1 (all uninitialized cells default to 0). The output of a NAND-TM program is the string `Y[0], ..., Y[m-1]` where m is the smallest integer such that `Y_nonblank[m]` $= 0$.

We now formally define a NAND-TM program.

Definition 6.4 (NAND-TM Programs)

A **NAND-TM program** consists of a sequence of lines of the form `foo = NAND(bar, blah)` and ending with a line of the form `MODANDJMP(foo, bar)`, where `foo, bar, blah` are either *scalar variables* (sequence of letters, digits, and underscores) or *array variables* of the form `Foo[i]` (starting with capital letters and indexed by `i`). The program has the array variables `X, X_nonblank`, `Y, Y_nonblank` and the index variables `i` built in, and can use additional array and scalar variables.
If P is a NAND-TM program and $x \in \{0,1\}^*$ is an input then an execution of P on x is the following process:

1. The arrays `X` and `X_nonblank` are initialized by `X[i]` $= x_i$ and `X_nonblank[i]` $= 1$ for all $i \in [|x|]$. All other variables and cells are initialized to 0. The index variable `i` is also initialized to 0.
2. The program is executed line by line. When the last line `MODANDJMP(foo, bar)` is executed we do as follows:
   (a) If `foo, bar` $= 1, 0$, jump to the first line without modifying the value of `i`.
   (b) If `foo, bar` $= 1, 1$, incremenet `i` by one and jump to the first line.
   (c) If `foo, bar` $= 0, 1$, then decrement `i` by one (unless it is already 0) and jump to the first

line.
(d) If `foo, bar` $= 0, 0$, halt and output `Y[0], ..., Y[m-1]` where m is the smallest integer such that `Y_nonblank[m]` $= 0$.

Here are some components of Turing machines and their analogs in NAND-TM programs.

1. The *state* of a Turing machine is equivalent to the *scalar-variables* such as `foo, bar, etc.`, each taking values in $\{0, 1\}$.
2. The *tape* of a Turing machines is equivalent to the *arrays*, where the component of each array is either 0 or 1.
3. The *head location* is equivalent to the *index variable*
4. *Accessing memory*: At every step the Turing machine has access to its local state, but can only access the tape at the position of the current head location. In a NAND-TM program, it has access to all the scalar variables, but can only access the arrays at the location `i` of the index variable.
5. A Turing machine can move the head location by at most one position in each step, while a NAND-TM program can modify the index `i` by at most one.

Theorem 6.1 (Equivalence of Turing Machines and NAND-TM programs)

For every function $F : \{0,1\}^* \longrightarrow \{0,1\}^*$, F is computable by a NAND-TM program P if and only if there is a Turing machine M that computes F.

Setting	Specification	Implentation
Finite Computation	$F : \{0,1\}^n \to \{0,1\}^m$	Circuit, Straightline program
Infinite Computation	$F : \{0,1\}^* \to \{0,1\}^*$	Algorithm, Turing Machine, Program

Finally, we can use syntactic sugar to make NAND-TM programs easier to write. For starters, we can use all of the syntactic sugar of NAND-CIRC, such as macro definitions and conditionals (if/then). However, we can go beyond this and achieve:

1. Inner loops such as the `while` and `for` operations common to many programming languages.
2. Multiple index variables (e.g. not just `i` but also `j, k`, etc.).
3. Arrays with more than one dimension (e.g., `Foo[i][j]`).

This means that the set of functions computable by NAND-TM with this feature is the same as the set of functions computable by standard NAND-TM.

6.1.1 Uniformity of Computation

Definition 6.5 (Uniformity)

The notion of a single algorithm that can compute functions of all input length is known as **uniformity** of computation.

Hence we think of Turing machines and NAND-TM as *uniform* models of computation, as opposed to Boolean circuits of NAND-CIRC, which are non-uniform models, in which we have to specify a different program for every input length. This uniformity leads to another crucial difference between Turing machines and circuits. Turing machines can have inputs and outputs that are longer than the description of the machine as a string, and in particular there exists a Turing machine that can "self replicate" in the sense that it can print its own code. This is extremely useful.

In summary, the main differences between uniform and non-uniform models are described as such:

1. **Non-uniform computational models**: Examples are NAND-CIRC programs and Boolean circuits. These are models where each individual program/circuit can compute a *finite* function

$$f : \{0,1\}^n \longrightarrow \{0,1\}^m \tag{80}$$

   We have seen that *every* finite function can be computed by *some* program/circuit. To discuss computation of an *infinite* function $F : \{0,1\}^* \longrightarrow \{0,1\}^*$, we need to allow a *sequence* $\{P_n\}_{n\in\mathbb{N}}$ of programs/circuits (one for every input length), but this does not capture the notion of a *single algorithm* to compute the function F.

2. **Uniform computational models**: Examples are Turing machines and NAND-TM programs. These are models where a single program/Turing machine can take inputs of *arbitrary length* and hence compute an *infinite* function

$$F : \{0,1\}^* \longrightarrow \{0,1\}^* \tag{81}$$

   The number of steps that a program/machine takes on some input is not a priori bounded in advance and in particular there is a chance that it will enter into an *infinite loop*. Unlike the non-uniform case, we have *not* shown that every infinite function can be computed by some NAND-TM program/Turing machine.

6.2 RAM Machines and NAND-RAM Programs

Note that since Turing machines (and NAND-TM programs) can only access one locations of arrays/tape at a time, they do not have *RAM*.

Definition 6.6 (RAM Machine)

The computational model that models access to such a memory is the **RAM machine**. The **memory** of a RAM machine is an array of unbounded size where each cell can store a single **word**, which can be thought of as a string in $\{0,1\}^\omega$ and also (equivalently) as a number in $[2^\omega]$.

For example, many modern computing architectures use 64-bit words, in which every memory location holds a string in $\{0,1\}^{64}$. The parameter ω is known as the *word size*. In addition to the memory array, a RAM machine also contains a constant number of **registers** $r_0, r_1, ..., r_{k-1}$, each of which can also contain a word.

The oeprations a RAM machine can carry out include:

1. **Data movement**: Load data from a certain cell in memory into a register or store the contents of a register into a certain cell of memory. A RAM machine can directly access any cell of memory without having to move the "head" (as Turing machines do) to that location. That is, in one step a RAM machine can load into register r_i the contents of the memory cell indexed by register r_j, or store into the memory cell indexed by register r_j the contents of register r_i.

2. **Computation**: RAM machines can carry out computation on registers such as arithmetic operations, logical operations, and comparisons.

3. **Control flow**: As in the case of Turing machines, the chose of what instruction to perform next can depend on the state of the RAM machine, which is captured by the contents of its register.

Just as the NAND-TM programming language models Turing machines, we can also define a **NAND-RAM programming language** that models RAM machines. The NAND-RAM programming language extends NAND-TM by adding the following features:

1. The variables of NAND-RAM are allowed to be (non-negative) *integer valued* rather than only Boolean. That is, a scalar variable `foo` holds a nonnegative integer in $\mathbb{N}$ and an array variable `Bar` holds an array of integers. As in the case of RAM machines, we will not allow integers of unbounded size.

2. We allow *indexed access* to arrays. If `foo` is a scalar and `Bar` is an array, then `Bar[foo]` refers to the location of `Bar` indexed by the value of `foo`. Note that this means that we don't need to have a special index variable `i` anymore.
3. We will assume that for Boolean operations such as `NAND`, a zero valued integer is considered as *false*, and a nonzero valued integer is considered as *true*.
4. In addition to `NAND`, NAND-RAM also includes all the basic arithmetic operations of addition, subtraction, multiplication, integer division, as well as comparisions (equal, greater/less than, etc.).
5. NAND-RAM includes conditional statements `if/then` as a part of the language.
6. NAND-RAM contains looping constructs such as `while` and `do` as part of the language.

It is easy to see that NAND-RAM programs are clearly more powerful than NAND-TM, and so if a function F is computable by a NAND-TM program then it can be computed by a NAND-RAM program. It turns out to be true that if a function is computable by a NAND-RAM program, then it can also be computed by a NAND-TM program.

Theorem 6.2 (Equivalence of Turing and RAM Machines)

Turing machines (aka NAND-TM programs) and RAM machines (aka NAND-RAM programs) are equivalent. That is, for every function

$$F : \{0,1\}^* \longrightarrow \{0,1\}^*, \tag{82}$$

F is computable by a NAND-TM program if and only if F is computable by a NAND-RAM program. Therefore, all four models are equivalent to one another.

7 Turing Completeness and Equivalence

Even though the notion of computing a function using Turing machines is crucial in theory, it is not a practical way of preforming computation. But in addition to defining computable functions with Turing machines, there are many equivalent conditions of computability under a wide variety of computational models. This notion is known as *Turing completeness* or *Turing equivalence*.

Any of the standard programming languages such as C, Java, Python, Pascal, Fortran, have very similar operations to NAND-RAM. Indeed, ultimately, they can all be executed by machines which have a fixed number of registers and a large memory array. Hence, with the equivalence theorem, we can simulate any program in such a programming language by a NAND-TM program. In the other direction, it is a fairly easy programming exercise to write an interpreter for NAND-TM in any of the above programming languages. Hence we can also simulate NAND-TM programs (and Turing machines) using these programming languages.

Definition 7.1 (Turing-Complete)

A computational system is said to be **Turing-complete** or **computationally universal** if it can be be used to simulate any Turing machine or NAND-TM.
Very much related, the property of being *equivalent* in power to Turing machines/NAND-TM is called **Turing equivalent**. That is, two computer P and Q are equivalent if P can simulate Q and Q can simulate P. All known Turing complete systems are Turing equivalent.

The equivalence between Turing machines and RAM machines allows us to choose the most convenient language for the task at hand:

1. When we want to *prove a theorem* about all programs/algorithms, we can use Turing machines (or NAND-TM) since they are simpler and easier to analyze.

2. If we want to show that a certain function *cannot* be computed, then we will use Turing machines.
3. When we want to show that a function can be computed we can use RAM machines or NAND-RAM, because they are easier to program in and correspond more closely to high level programming languages we are used to. In fact, we will often describe NAND-RAM programs in an informal manner, trusting that the reader can fill in the details and translate the high level description to the precise program. (This is just like the way people typically use informal or "pseudocode" descriptions of algorithms, trusting that their audience will know to translate these descriptions to code if needed.)

A formal definition of Turing completeness is as follows. This is also referred to as *Godel Numbering*, which is a function that assigns to each symbol and well-formed formula of some formal language a unique natural number, called its Gödel number

Definition 7.2 (Turing Completeness and Equivalence)

Let $\mathcal{F}$ be the set of all partial functions from $\{0,1\}^*$ to $\{0,1\}^*$. A **computational model** is a map

$$\mathcal{M} : \{0,1\}^* \longrightarrow \mathcal{F} \tag{83}$$

We say that a program $P \in \{0,1\}^*$ $\mathcal{M}$**-computes** a function $F \in \mathcal{F}$ if

$$\mathcal{M}(P) = F \tag{84}$$

A computational model $\mathcal{M}$ is **Turing complete** if there is a computable map

$$ENCODE_{\mathcal{M}} : \{0,1\}^* \longrightarrow \{0,1\}^* \tag{85}$$

such that for every Turing machine N (represented as a string), $\mathcal{M}(ENCODE_{\mathcal{M}}(N))$ is equal to the partial function computed by N.
A computational model $\mathcal{M}$ is **Turing equivalent** if it is Turing complete and there exists a computable map $DECODE_{\mathcal{M}} : \{0,1\}^* \longrightarrow \{0,1\}^*$ such that for every string $P \in \{0,1\}^*$, $N = DECODE_{\mathcal{M}}(P)$ is a string representation of a Turing machine that computes the function $\mathcal{M}(P)$.

7.1 Cellular Automata

Many physical systems can be described as consisting of a large number of elementary components that interact with one another. One way to model such systems is using cellular automata. This is a system that consists of a large (or even infinite) number of cells. Each cell only has a constant number of possible states. At each time step, a cell updates to a new state by applying some simple rule to the state of itself and its neighbors.

Definition 7.3 (Conway's Game of Life)

An example of a cellular automaton is **Conway's Game of Life**. In this automata the cells are arranged in an infinite two dimensional grid. Each cell has only two states:
1. Dead: which we encode as a 0
2. Alive: which we encode as 1

The next state of a cell depends on its previous state and the states of its 8 adjacent neighbors, which can be modeled with a transition function

$$r : \Sigma^8 \longrightarrow \Sigma \tag{86}$$

A dead cell becomes alive only if exactly three of its neighbors are alive. A live cell continues to live if it has two or three live neighbors.

Even though the number of cells is potentially infinite, we can encode the state using a finite-length string by only keeping track of the live cells. If we initialize the system in a configuration with a finite number of live cells, then the number of live cells will stay finite in all future steps. Note that this is a discrete time Markov chain.

Since the cells in the game of life are arranged in an infinite two-dimensional grid, it is an example of a *two dimensional cellular automaton*. We can get even simpler by setting a *one dimensional cellular automaton*, where the cells are arranged in an infinite line.

Theorem 7.1 (Conway's Game of Life Is Turing Complete)

Conway's Game of Life is Turing complete.

7.1.1 One-Dimensional Cellular Automata

Definition 7.4 (One-Dimensional Cellular Automaton)

Let $\Sigma = \{0, 1, \emptyset\}$. A **one-dimensional cellular automaton** of alphabet Σ is described by a *transition rule*

$$r : \Sigma^3 \longrightarrow \Sigma \tag{87}$$

A **configuration** of the automaton r is a function $A : \mathbb{Z} \longrightarrow \Sigma$; that is, A just represents an infinite sequence of letters in the alphabet Σ. If an automaton with rule r is in configuration A, then its next configuration $A' = NEXT_r(A)$, is the function A' such that

$$A'(i) = r\big(A(i-1), A(i), A(i+1)\big) \tag{88}$$

In other words, the next state of the automaton r at point i is obtained by applying the rule r to the values of A at i and its two neighbors.
It is also said that a configuration of an automaton r is **finite** if there is only some finite number of indices $i_0, ..., i_{j-1}$ in $\mathbb{Z}$ such that $A(i_j) \neq \emptyset$.

If the alphabet is only $\{0, 1\}$, then there can be a total of $2^8 = 256$ total possible one dimensional cellular automata. For example, the cellular automaton with the transition rule

$$r(L, C, R) \equiv C + R + CR + LCR \pmod 2 \tag{89}$$

can be expressed with the table (called rule 110)

111	110	101	100	011	010	001	000
0	1	1	0	1	1	1	0

However, many of them are trivially equivalent to each other up to a simple transformation of the underlying geometry, such as with reflections, translations, or rotations. This reduces the possible unique automata to 88, only one of which is Turing complete.

Theorem 7.2 (Rule 110 Is Turing Complete)

The Rule 110 cellular automaton is Turing complete. That is, any calculation or computer program can be simulated using this automaton.

Definition 7.5 (Configuration of Turing Machines)

Let M be a Turing machine with tape alphabet Σ and state space $[k]$. A **configuration** of M is a string

$$\alpha \in \overline{\Sigma}^*, \text{ where } \overline{\Sigma} = \Sigma \times \big(\{\cdot\} \cup [k]\big) \tag{90}$$

that satisfies that there is exactly one coordinate i for which $\alpha_i = (\sigma, s)$ for some $\sigma \in \Sigma$ and $s \in [k]$. For all other coordinates j, $\alpha_j = (\sigma', \cdot)$ for some $\sigma' \in \Sigma$. A configuration of $\alpha \in \overline{\Sigma}^*$ of M corresponds to the following staet of its execution:

1. M's tape contains $\alpha_{j,0}$ for all $j < |\alpha|$ and contains $\emptyset$ for all positions that are at least $|\alpha|$, where we let $\alpha_{j,0}$ be the value σ such that $\alpha_j = (\sigma, t)$ with $\sigma \in \Sigma$ and $t \in \{\cdot\} \cup [k]$. In other words, since α_j is a pair of an alphebet symbol σ and either a state in $[k]$ or the symbol $\cdot$, $\alpha_{j,0}$ is the first component σ of this pair.
2. M's head is in the unique position i for which α_i has the form (σ, s) for $s \in [k]$, and M's state is equal to s.

Informally, a configuration can be interpreted simply as a string that encodes a *snapshot* of the Turing machine at a given point in the execution. It is also called a *core dump*. Such a snapshot must encode the following components:

1. The current head position.
2. The full contents of the large scale memory, that is the tape.
3. The contents of the "local registers," that is the state of the machine.

7.2 Lambda Calculus

The **Lambda calculus** is an abstract mathematical theory of computation, involving λ functions. It is a Turing complete language. λ calculus allows us to define "anonymous" functions. For example, instead of giving a name f to a function and defining it as

$$f(x) = x^2 \tag{91}$$

we can write it anonymously (without naming it at all) as

$$x \mapsto x^2, \text{ or equivalently, } \lambda x.x^2 \tag{92}$$

so $(\lambda x.x^2)(7) = 49$, or by dropping the parentheses, $(\lambda x.x^2)7 = 49$. That is, we can interpret $\lambda x.exp(x)$, where exp is some expression as a way of specifying the anonymous function $x \mapsto exp(x)$. This notation occurs in many programming languages, such as Python, where the squaring function is written `lambda x: x*x`.

Furthermore, in λ calculus functions are *first-class objects*, meaning that we can use functions as arguments to other functions. However, *all functions must take one input.*

Expressions can be thought of as programs in the language of lambda calculus. Given the notion of a variable, denoted by $x, y, z, ...$ we recursively define an expression inductively in terms of abstractions (anonymous functions) and applications as follows:

Definition 7.6 (λ expression)

Let Λ be the set of λ expressions. Then

1. Identifier: If x is a variable, then $x \in \Lambda$
2. Abstractions: If x is a variable and $\mathcal{M} \in \Lambda$, then $(\lambda x.\mathcal{M}) \in \Lambda$
3. Applications: If $\mathcal{M} \in \Lambda$ and $\mathcal{N} \in \Lambda$, then $\mathcal{M}\,\mathcal{N} \in \Lambda$
4. Grouping: If $\mathcal{M}$ is an expression, then $(\mathcal{M}) \in \Lambda$

Here are two important conventions:

1. Function application is left associative, unless stated otherwise by parentheses:

$$\mathcal{S}_1\mathcal{S}_2\mathcal{S}_3 \equiv \big((\mathcal{S}_1\mathcal{S}_2)\mathcal{S}_3\big) \tag{93}$$

2. Consecutive abstractions can be uncurried, e.g.

$$\lambda xyz.\mathcal{M} \equiv \lambda x.\lambda y.\lambda z.\mathcal{M} \tag{94}$$

3. The body of the abstraction extends to the right as far as possible

$$\lambda x.\mathcal{M}\,\mathcal{N} \equiv \lambda x.(\mathcal{M}\,\mathcal{N}) \tag{95}$$

7.2.1 Applications

The notation for applying a function to a certain input is modeled by juxtaposition. That is,

$$f(a) \implies f\ a \tag{96}$$

where $f\ a$ means the function f *applied on input* a. However, since functions themselves could be inputs and outputs to other functions, we can use a method called **currying** to create multivariate functions. In the one below,

$$f\ a\ b \text{ , which stands for } f(a)(b) \tag{97}$$

this does not model a multivariate function f that takes two inputs. Rather, f takes one input a and outputs a function that takes one input b!

Example 7.1 (Addition in the Lambda Calculus)

The addition function `add(a)(b)` can be modeled with 2 steps.

1. It takes the first argument a and outputs a function `adda` that takes another argument.

$$\texttt{add} : a \mapsto \texttt{adda} \tag{98}$$

2. `adda` takes argument b and adds b to the predetermined number a.

$$\texttt{adda} : b \mapsto a + b \tag{99}$$

Additionally, the expression

$$(f\ a)\ b \text{ , which stands for } \big(\text{f(a)}\big)\text{(b)} \tag{100}$$

is equivalent to $f\ a\ b$ since we have stated that function application is left associative. However,

$$f\ (a\ b) \text{ , which stands for f } \big(\text{a(b)}\big) \tag{101}$$

is a different expression, since now we are applying a onto b first, getting the output, and then applying f onto the output.

For example

$$((\lambda x.(\lambda y.x))2)9 = (\lambda y.2) = 9$$

Using a method called **currying**, we can actually create multivariate functions. For example, the function

$$\lambda x.(\lambda y.x + y) \tag{102}$$

maps x to the function $y \mapsto x + y$, which is equivalent to a function mapping $(x, y) \mapsto x + y$.

7.2.2 Abstractions

To understand abstractions, observe the four examples below (where $\implies$ means mapped to).

$$\begin{aligned} \lambda\, a.b \quad & a \implies b \\ \lambda\, a.b\, x \quad & a \implies b(x) \\ \lambda\, a.(b\, x) \quad & a \implies \big(b(x)\big) \\ (\lambda\, a.b)\, x \quad & (a \implies b)(x) \end{aligned}$$

In the second example, note that since the body of the abstraction extends to the far right as possible (i.e. the λ abstraction is greedy), it outputs the entire $b\, x$. The extra parentheses in the third line is not needed because of this convention. However, the parentheses in the fourth line is nontrivial. It says that $\lambda\, a.b$ outputs a function that acts on x. Finally, we are allowed to nest functions as such:

$$\lambda\, a.\lambda\, b.a \qquad a \implies b \implies a \tag{103}$$

The outermost λ takes in an a and returns a function that takes in a b, which in turn outputs the a. Note that $\lambda\, a.\lambda\, b.a = \lambda\, a.(\lambda\, b.a)$.

7.2.3 Beta Reduction

β-reduction refers to the process in simplifying a λ expression.

Example 7.2 (Beta Normal Form)

We can β reduce the expression into its simplest form, called the **beta normal form**.

$$\begin{aligned} ((\lambda\, a.a)\ \lambda\, b.\lambda\, c.b)(x)\ \lambda\, e.f &= (\lambda\, b.\lambda\, c.b)(x)\ \lambda e.f \\ &= (\lambda\, c.x)\ \lambda\, e.f \\ &= x \end{aligned}$$

7.2.4 Combinators

Like transistors and Boolean gates, combinators are the atoms of more complicated functions in lambda calculus. We list five of them. Note that the cardinal can be build from other combinators.

Smyb	Bird	λ-Calculus	Use
I	Idiot	$\lambda\, a.a$	identity
M	Mockingbird	$\lambda\, f.ff$	self-application
K	Kestrel	$\lambda\, ab.a$	first, const
KI	Kite	$\lambda\, ab.b = KI = CK$	second
C	Cardinal	$\lambda\, fab.fba$	reverse arguments

7.2.5 Free and Bound Variables

In an abstraction like $\lambda x.x$, the variable x is something that has no original meaning but is a placeholder (i.e. it only has meaning within the λ function). We say that x is a variable **bound** to the λ. On the other hand, in $\lambda x.y$ i.e. a function which always returns y whatever it takes, y is a free variable since it has an independent meaning by itself. Because a variable is bound in some sub-expression does not mean it is bound everywhere. For example, the following is a valid expression (an example of application)

$$(\lambda x.x)(\lambda y.yx) \tag{104}$$

Here, the x in the second parenthesis has nothing to do with the one in the first. Formally,

Definition 7.7 (Free Variables of a Lambda Expression)

x is free...
1. in the expression x
2. in the expression $\lambda y.\mathcal{M}$ if $x \neq y$ and x is free in $\mathcal{M}$
3. in $\mathcal{M}\,\mathcal{N}$ if x is free in $\mathcal{M}$ or if it is free in $\mathcal{N}$

x bound...
1. in the expression $\lambda x.\mathcal{M}$
2. in $\mathcal{M}\,\mathcal{N}$ if x is bound in $\mathcal{M}$ or if it is bound in $\mathcal{N}$

Note that a variable can be both bound and free but they represent different things. An expression with no free variables is called a **closed expression**.

In addition, the concept of α equivalence states that any bound variable is a placeholder and can be replaced with a different variable, provided there are no clashes. A simple example is

$$\lambda x.x =_\alpha \lambda y.y \tag{105}$$

However,

$$\lambda x.(\lambda x.x) =_\alpha \lambda y.(\lambda x.x) \text{ but not to } \lambda y.(\lambda x.y) \tag{106}$$

Example 7.3 (Simplifying a Lambda Expression)

The following λ expression can be simplified as such:

$$\big(\lambda x.(\lambda x.x)\big)y =_\alpha \lambda y.y =_\alpha \lambda x.x \tag{107}$$

7.2.6 Booleans as Functions

Note that we can now define Booleans as functions! We can define a function f that outputs, one element if it is the True function and outputs another element if it is the False function. This can be done by defining:

$$T(a,b) = \lambda x.\lambda y.x(a)(b) = a \text{ (the Kestrel!)}$$
$$F(a,b) = \lambda x.\lambda y.y(a)(b) = b \text{ (the Kite!)}$$

Similarly, we can define the not function using the Cardinal.

Symb	Name	λ-Calculus	Use
T	True	$\lambda\, ab.a = K$	encoding for True
F	False	$\lambda\, ab.b$	encoding for False
	Not	$\lambda\, p.pFT = C$	negation

It is easy to see C as the negation function since

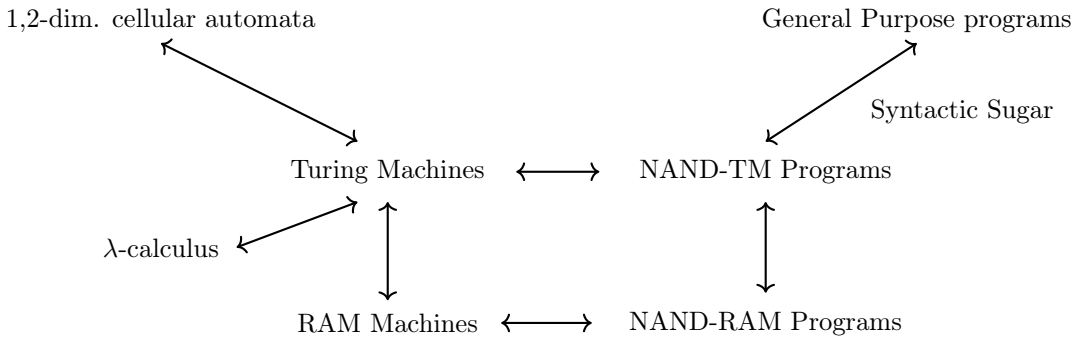
$$K(a)(b) = a \implies CK(a)(b) = b$$
$$KI(a)(b) = b \implies CKI(a)(b) = a$$

With this, we can build more complex logic gates, making the lambda calculus equivalent in computing power to NAND-CIRC programs. Similarly, we can cleverly implement recursion and arrays into this language, therefore making the lambda calculus Turing complete. To implement infinite loops, consider the λ expression

$$\lambda x.xx\ \lambda x.xx \tag{108}$$

If we try to simply this expression by invoking the left hand function on the right one, then we just get another copy of this expression.

The Turing equivalence of the computing models we have talked about can be visualized below:



8 Universality

It turns out that uniform models such as Turing machines or NAND-TM programs allow us to obtain a truly *universal Turing machine* U that can evaluate all other machines, including machines that are more complex than U itself. Similarly, there is a *Universal NAND-TM program* U' that can evaluate all NAND-TM programs, including programs that have more lines than U'.

The existence of such a universal program/machine underlies the technological advances made up to now. Rather than producing special purpose calculating devices such as the abacus, the slide ruler, and machines that compute various trigonometric series, this universal property allows us to build a machine that, via software, can be extended to do arbitrary computations, i.e. a *general purpose computer*.

Theorem 8.1 (Universal Turing Machine)

There exists a Turing machine U such that on every string M which represents a Turing machine and $x \in \{0,1\}^*$,

$$U(M,x) = M(x) \tag{109}$$

That is, if the machine M halts on x and outputs some $y \in \{0,1\}^*$, then $U(M,x) = y$ and if M does not halt on x (i.e. $M(x) = \perp$), then $U(M,x) = \perp$.

There is more than one Turing machine U that satisfies the theorem above.

Definition 8.1 (String representation of Turing machine)

Let M be a Turing machine with k states and size l alphabet

$$\Sigma = \{\sigma_0, \sigma_1, ..., \sigma_{l-1}\} \tag{110}$$

(We use the convention $\sigma_0 = 0, \sigma_1 = 1, \sigma_2 = \emptyset, \sigma_3 = \triangleright$. We represent M as the triple (k, l, T), where T is the table of valies for δ_M:

$$T = \big(\delta_M(0,\sigma_0), \delta_M(0,\sigma_1), ..., \delta_M(k-1,\sigma_{l-1})\big) \tag{111}$$

where each value $\delta_M(s,\sigma)$ is a triple (s',σ',d) with $s' \in [k], \sigma' \in \Sigma$, and d a number in $\{0,1,2,3\}$ encoding one of $\{\mathsf{L},\mathsf{R},\mathsf{S},\mathsf{H}\}$. Thus, such a machine M is encoded by a list of $2+3k\cdot l$ natural numbers. The **string representation** of M is obtained by concatenating prefix-free representations of all these integers. If a string $\alpha \in \{0,1\}^*$ does not represent a list of integers in the form above, then we treat it as representing the trivial Turing machine with one state that immediately halts on every input.

The big takeways so far are:

1. We can represent every Turing machine as a string.

2. Given the string representation of a Turing machine M and an input x, we can simulate M's execution on the input x. That is, if we want to simulate a new Turing machine M, we do not need to build a new physical machine, but rather can represent M as a string (i.e. using code) and then input M to the universal machine U.

9 Time Complexity

The Big O notation is a mathematical notation that describes the limiting behavior of a function when the argument tends towards a particular value of infinity. That is, if the time it takes for an algorithm to complete a problem with input size n is given by $f(n)$, then we say that the computational complexity is of the *order* $O(f(n))$. More formally, we can define it as such:

Definition 9.1 (Big-O Notation)

Let f and g be (nonnegative) real-valued functions both defined on the positive integers, and let $g(x)$ be strictly positive for all large enough values of x. One writes

$$f(x) = O\big(g(x)\big) \text{ as } x \to \infty \tag{112}$$

if the absolute value of $f(x)$ is at most a positive constant multiple of $g(x)$ for all sufficiently large values of x. That is, $f(x) = O\big(g(x)\big)$ if there exist positive integers M and n_0 such that

$$f(n) \leq Mg(n) \text{ for all } n \geq n_0 \tag{113}$$

In many contexts, the assumption that we are interested in the growth rate as the variable x goes to infinity is left unstated, and one write more simply that

$$f(x) = O\big(g(x)\big) \tag{114}$$

Example 9.1 (Simple Runtime Calulation for Polynomials)

Let there be a program that given input with length x, takes $f(x) = 6x^4 - 2x^3 + 5$ steps to solve whatever problem needs to be solved. Then, using the simplification steps above, we have

$$f(x) = O(x^4) \tag{115}$$

When talking about running time, what we care about is the *scaling behavior* of the number of steps as the input size grows (as opposed to a fixed number).

9.1 Formally Defining Running Time

We can informally define what it means for a function $F : \{0,1\}^* \longrightarrow \{0,1\}^*$ to be *computable* in time $T(n)$ steps, where T is some function mapping the length n of the input to the number of computation steps allowed.

Definition 9.2 (Computable in Turing Machine Time $T(n)$)

Let $T : \mathbb{N} \longrightarrow \mathbb{N}$ be some function. We say that a function $F : \{0,1\}^* \longrightarrow \{0,1\}^*$ is **computable in $T(n)$ Turing Machine time (TM-time for short)** if there exists a Turing machine M such that for every sufficiently large n and every $x \in \{0,1\}^n$, the machine halts after executing at most $T(n)$ steps and outputs $F(x)$.
We define $TIME_{\mathsf{TM}}\big(T(n)\big)$ to be the set of Boolean functions ($\{0,1\}^* \longrightarrow \{0,1\}$) that are computable in $T(n)$ TM time. Note that $TIME_{\mathsf{TM}}\big(T(n)\big)$ is a class of *functions*, not machines.

With this, we can formally define what is means for function $F : \{0,1\}^* \longrightarrow \{0,1\}$ to be computable in time at most $T(n)$ where n is the size of the input. Furthermore, the property of considering only "sufficiently large" n's is not very important but it is convenient since it allows us to avoid dealing explicitly with

uninteresting "edge cases." We have also defined computability with Boolean functions for simplicity, but we can generalize this further.

9.1.1 Polynomial and Exponential Time

Definition 9.3 (Decision Problem)

A **decision problem** is a problem that can be posed as a yes-no question on an infinite set of inputs. A method for solving a decision problem, given in the form of an *algorithm*, is called a **decision procedure** for that problem. A decision problem which can be solved by an algorithm is called **decidable**.

It is traditional to define the decision problem as the set of possible inputs together with the set of inputs for which the answer is yes, and the set of inputs (i.e. the domain) can be numbers, floats, strings, etc.

Example 9.2 (Examples of Decision Problems)

Two examples of decision problems are:
1. Deciding whether a given natural number is prime.
2. Given two numbers x and y, does x evenly divide y? The decision procedure can be long division.

Definition 9.4 (Polynomial and Exponential Time)

The two main time complexity classes are defined:
1. **Polynomial time**: A function $F : \{0,1\}^* \longrightarrow \{0,1\}$ is **computable in polynomial time** if it is in the class
$$\mathbf{P} = \bigcup_{c \in \{1,\dots m\}} TIME_{\mathsf{TM}}\big(n^c\big), \quad m \in \mathbb{N} \tag{116}$$
That is, $F \in \mathbf{P}$ if there is an algorithm to compute F that runs in time at most *polynomial* in the length of the input.
2. **Exponential time**: A function $F : \{0,1\}^* \longrightarrow \{0,1\}$ is **computable in exponential time** if it is in the class
$$\mathbf{EXP} = \bigcup_{c \in \{1,\dots,m\}} TIME_{\mathsf{TM}}\big(2^{n^c}\big) \tag{117}$$
That is, $F \in \mathbf{EXP}$ if there is an algorithm to compute F that runs in time at most *exponential* in the length of the input.

Summarizing this, we say that $F \in \mathbf{P}$ if there is a polynomial $p : \mathbb{N} \longrightarrow \mathbb{R}$ and a Turing machine M such that for every $x \in \{0,1\}^*$, when given input x, the Turing machine halts within at most $p(|x|)$ steps and outputs $F(x)$.
We say that $F \in \mathbf{EXP}$ if there is a polynomial $p : \mathbb{N} \longrightarrow \mathbb{R}$ and a Turing machine M such that for every $x \in \{0,1\}^*$, when given input x, M halts within at most $2^{p(|x|)}$ steps and outputs $F(x)$.

Lemma 9.1 ($\mathbf{P} \subseteq \mathbf{EXP}$)

Since exponential time is much larger than polynomial time,
$$\mathbf{P} \subset \mathbf{EXP} \tag{118}$$

Time complexity for the previous algorithms are as follows:

P	**EXP** (not known to be **P**)
Shortest path	Longest path
Min cut	Max cut
2SAT	3SAT
Linear eqs	Quad eqs
Zerosum	Nash
Determinant	Permanent
Primality	Factoring

Many technological developments are centered around these facts. For example, the exponential time complexity of factoring algorithms is what makes the RSA-encryption so secure. If a polynomial time algorithm for factoring were to be discovered, RSA-encryption would be rendered obsolete.

9.2 Modeling Running Time Using RAM Machines/NAND-RAM

Despite the theoretical elegance of Turing machines, RAM machines and NAND-RAM programs are much more closely related to actual computing architecture. For example, even a "merge sort" program cannot be implemented on a Turing machines in $O(n \log n)$ time. We can define running time with respect to NAND-RAM programs just as we did for Turing machines.

Definition 9.5 (Computable in RAM Time $T(n)$)

Let $T : \mathbb{N} \longrightarrow \mathbb{N}$. We say that a function $F : \{0,1\}^* \longrightarrow \{0,1\}^*$ is **computable in T(n) RAM time (RAM-time for short)** if there exists a NAND-RAM program P such that for every sufficiently large n and every $x \in \{0,1\}^n$, when given input x, the program P halts after executing at most $T(n)$ lines and outputs $F(x)$.
We define $TIME_{\mathsf{RAM}}\big(T(n)\big)$ to be the set of Boolean functions ($\{0,1\}^* \longrightarrow \{0,1\}$) that are computable in $T(n)$ RAM time.

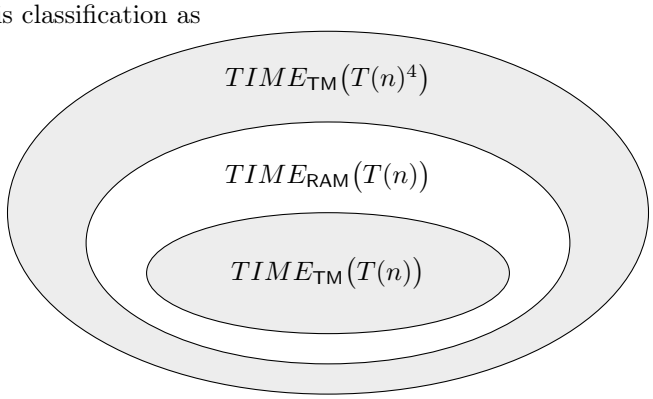
We will use $TIME\big(T(n)\big)$ to denote $TIME_{\mathsf{RAM}}\big(T(M)\big)$. However, as long as we only care about the difference between exponential and polynomial time, the model of running time we use does not make much difference. The reason is that Turing machines can simulate NAND-RAM programs with at most a polynomial overhead.

Theorem 9.2 (Relating RAM and Turing machines)

Let $T : \mathbb{N} \longrightarrow \mathbb{N}$ be a function such that $T(n) \geq n$ for every n and the map $n \mapsto T(n)$ can be computed by a Turing machine in time $O(T(n))$. Then,

$$TIME_{\mathsf{TM}}\big(T(n)\big) \subseteq TIME_{\mathsf{RAM}}\big(10 \cdot T(n)\big) \subseteq TIME_{\mathsf{TM}}\big(T(n)^4\big) \tag{119}$$

We can visually see this classification as



With this, we could have equally defined **P** as the class of functions computable by NAND-RAM programs (instead of Turing machines) that run in polynomial time in the length of the input. Similarly, with $T(n) = 2^{n^a}$, we see that the class **EXP** can also be defined as the set of functions computable by NAND-RAM programs in time at most $2^{p(n)}$ where p is some polynomial. This justifies the choice of **P** as capturing a technology-independent notion of tractability. Therefore, *all "reasonable" computational models are equivalent if we only care about the distinction between polynomial and exponential*, with reasonable referring to all scalable computational models that have been implemented except possibly quantum computers.

When considering general time bounds, we need to make sure to rule out some "exceptional" cases such as functions T that don't give enough time for the algorithm to even read the input, or functions where the time bound itself is uncomputable. More precisely, T must be a *nice function*.

Definition 9.6 (Nice Time Bound Function)

That is why we say that the function $T : \mathbb{N} \longrightarrow \mathbb{N}$ is a **nice time bound function** (**nice function** for short) if

1. for every $n \in \mathbb{N}$ $T(n) \geq n$ (T allows enough time to read the input)
2. for every $n' \geq n$, $T(n') \geq T(n)$ (T allows more time on longer inputs)
3. the map $F(x) = 1^{T(|x|)}$ (i.e. mapping a string of length n to a sequence of $T(n)$ ones) can be computed by a NAND-RAM program in $O(T(n))$ time

So, the following are examples of polynomially equivalent models:

1. Turing machines
2. NAND-RAM programs/RAM machines
3. All standard programming languages, including C/Python/Javascript...
4. The λ calculus
5. Cellular automata
6. Parallel computers
7. Biological computing devices such as DNA-based computers

The *Extended Church Turing Thesis* is the statement that this is true for all physically realizable computing models. In other words, the extended Church Turing thesis says that for every *scalable computing device* C (which has a finite description but can be in principle used to run computation on arbitrarily large inputs), there is some constant a such that for every function $F : \{0,1\}^* \longrightarrow \{0,1\}$ that C can compute on n length inputs using an $S(n)$ amount of physical resources. This is a strengthening of the plain Church Turing Thesis, which states that the set of computable functions is the same for all physically realizable models, but without requiring the overhead in the simulation between different models to be at most polynomial.

Like the Church-Turing thesis itself, the extended Church-Turing thesis is in the asymptotic setting and does not directly yield an experimentally testable prediction. However, it can be instantiated with more concrete bounds on the overhead, yielding experimentally- testable predictions such as the Physical Extended Church-Turing Thesis.

9.3 Efficient Universal Machine: A NAND-RAM Interpreter in NAND-RAM

We can now see that the universal Turing machine U, which can compute every Turing machine M, has a *polynomial* overhead for simulating a $NAND - TM$ program. That is, it can simulate T steps of a given $NAND - TM$ (or $NAND - RAM$) program P on an input x in $O(T^4)$ steps. But in fact, by directly simulating $NAND - RAM$ programs we can do better with only a *constant* multiplicative overhead.

Theorem 9.3 (Efficient Universality of NAND-RAM)

There exists a NAND-RAM program U satisfying the following:

1. *U is a universal NAND-RAM program*: For every NAND-RAM program P and input x, $U(P,x) = P(x)$ where by $U(P,x)$ we denote the output of U on a string encoding the pair (P,x).
2. *U is efficient*: There are some constants a, b such that for every $NAND-RAM$ program P, if P halts on input x after most T steps, then $U(P,x)$ halts after at most $C \cdot T$ steps where $C \leq a|P|^b$.

This leads to a corollary. Given any Turing machine M, input x, and *step budget* T, we can simulate the execution for M for T steps in time that is polynomial in T. Formally, we define a function $TIMEDEVAL$ that takes the three parameters M, x, and the time budget, and outputs $M(x)$ if M halts within at most T steps, and outputs 0 otherwise. That is, let $TIMEDEVAL : \{0,1\}^* \longrightarrow \{0,1\}^*$ be the function defined as

$$TIMEDEVAL(M, x, 1^T) = \begin{cases} M(x) & M \text{ halts within } \leq T \text{ steps on } x \\ 0 & \text{else} \end{cases} \tag{120}$$

Then, $TIMEDEVAL \in \mathbf{P}$, i.e. the timed universal Turing machine computes $TIMEDEVAL$ in polynomial time.

9.4 The Time Hierarchy Theorem

Some functions are uncomputable, but are there functions that can be computed, but only at an exorbitant cost? For example, is there a function that *can* be computed in time 2^n, but *cannot* be computed in time $2^{0.9n}$? It turns out that the answer is yes.

Theorem 9.4 (Time Hierarchy Theorem)

For every nice function $T : \mathbb{N} \longrightarrow \mathbb{N}$, there is a function $F : \{0,1\}^* \longrightarrow \{0,1\}$ in

$$TIME\big(T(n) \log n\big) \setminus TIME\big(T(n)\big) \tag{121}$$

There is nothing special about $\log n$. We could have used any other efficiently computable function that ends to infinity with n.

9.5 Non-Uniform Computation

10 Polynomial-Time Reductions

Let us redefine some of the problems into *decision problems*.

3SAT The *3SAT problem* can be phrased as the function $3SAT : \{0,1\}^* \longrightarrow \{0,1\}$ that takes as an input a 3CNF formula φ (i.e. a formula of the form $C_0 \wedge ... \wedge C_{m-1}$ where each C_i of the OR of three iterables) and maps φ to 1 if there exists some assignment to the variables of φ that causes it to evaluate to *true* and to 0 otherwise. For example,

$$3SAT\big((x_0 \vee \overline{x_1} \vee x_2) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (\overline{x_0} \vee \overline{x_2} \vee x_3)\big) = 1 \tag{122}$$

since the assignment $x = 1101$ satisfies the input formula.

Quadratic Equations The *quadratic equations problem* corresponds to the function $QUADEQ : \{0,1\}^* \longrightarrow \{0,1\}$ that maps a set of quadratic equations E to 1 if there is an assignment x that satisfies all equations and to 0 otherwise.

Longest Path The *longest path problem* correpsonds to the function $LONGPATH : \{0,1\}^* \longrightarrow \{0,1\}^*$ that maps a graph G and a number k to 1 if there is a simple path in G of length at least k, and maps (G,k) to 0 otherwise.

Maximum Cut The *maximum cut problem* corresponds to the function $MAXCUT : \{0,1\}^* \longrightarrow \{0,1\}$ that maps a graph G and a number k to 1 if there is a cut in G that cuts at least k edges, and maps (G,k) to 0 otherwise.

All of these problems above are in **EXP** but it is not known whether or not they are in **P**. However, we can reduce these problems to ones that are in **P**, proving that they are indeed in **P**.

10.1 Polynomial-Time Reductions

Suppose that that $F, G : \{0,1\}^* \longrightarrow \{0,1\}$ are two Boolean functions. A *polynomial-time reduction* (or *reduction*) from F to G is a way to sho that F is "no harder" than G in the sense that a polynomial-time algorithm for G implies a polynomial-time algorithm for F.

Definition 10.1 (Polynomial-time reductions)

Let $F, G : \{0,1\}^* \longrightarrow \{0,1\}$. We say that **F reduces to G**, denoted by $F \leq_p G$, if there is a polynomial-time computable $R : \{0,1\}^* \longrightarrow \{0,1\}^*$ such that for every $x \in \{0,1\}^*$,

$$F(x) = G\big(R(x)\big) \tag{123}$$

We say that F and G have **equivalent complexity** if $F \leq_p G$ and $G \leq_p F$. Clearly, $\leq_p$ is a transitive property.

10.2 Reducing 3SAT to Zero-One and Quadratic Equations

Definition 10.2 (Zero-One Linear Equations Problem)

The **Zero-One Linear Equations problem** corresponds to the function

$$01EQ : \{0,1\}^* \longrightarrow \{0,1\} \tag{124}$$

whose input is a collection E of linear equations in variables $x_0, ..., x_{n-1}$, and the output is 1 iff there is an assignment $x \in \{0,1\}^n$ satisfying the matrix equation

$$Ax = b, \quad A \in \text{Mat}(m \times n, \{0,1\}), b \in \mathbb{N}^m \tag{125}$$

For example, if E is a string encoding the set of equations

$$\begin{aligned} x_0 + x_1 + x_2 &= 2 \\ x_0 + x_2 &= 1 \\ x_1 + x_2 &= 2 \end{aligned}$$

then $01EQ(E) = 1$ since the assignment $x = 011$ satisfies all three equations.

Note that if we extended the field to $\mathbb{R}$, then this can be solved using Gaussian elimination in polynomial time, but there is no known efficiently algorithm to solve $01EQ$. This is stated in the following theorem.

Theorem 10.1 (Hardness of 01 Linear Equations)

$$3SAT \leq_p 01EQ \tag{126}$$

This means that finding an efficient algorithm to solve $01EQ$ would imply an algorithm for $3SAT$. We can further use this to reduce $3SAT$ to the quadratic equations problem, where $QUADEQ(p_0, ..., p_{m-1}) = 1$ if

and only if there is a solution $x \in \mathbb{R}^n$ to the equations $p_i(x) = 0$ for $i = 0, ..., m-1$. For example, the following is a set of quadratic equations over the variables x_0, x_1, x_2:

$$\begin{aligned} x_0^2 - x_0 &= 0 \\ x_1^2 - x_1 &= 0 \\ x_2^2 - x_2 &= 0 \\ 1 - x_0 - x_1 + x_0 x_1 &= 0 \end{aligned}$$

Theorem 10.2 (Hardness of Quadratic Equations)

$$3SAT \leq_p QUADEQ \tag{127}$$

10.3 Independent Set and Other Graph Problems

Definition 10.3 (Independent Set)

For a graph $G = (V, E)$, an **independent set**, also known as a **stable set**, is a subset $S \subseteq V$ such that there are no edges with both endpoints in S (in other words, $E(S, S) = \emptyset$). Trivially, every singleton (of one point) is an independent set.

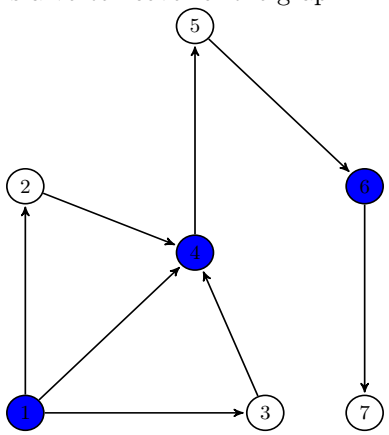
The **maximum independent set** problem is the task of finding the largest independent set in the graph. The independent set problem is naturally related to *scheduling problem*: if we put an edge between two conflicting tasks, then an independent set corresponds to a set of tasks that can all be scheduled together without conflicts.

Theorem 10.3 (Hardness of Independent Set)

$$3SAT \leq_p ISET \tag{128}$$

Definition 10.4 (Vertex Cover)

A **vertex cover** in a graph $G = (V, E)$ is a subset $S \subseteq V$ of vertices that touches all edges of G. For example, the following blue nodes is a vertex cover of the graph.



The **vertex cover problem** is the task to determine, given a graph G and a number k, whether there

exists a vertex cover in the graph with at most k vertices. Formally, this is the function

$$VC : \{0,1\}^* \longrightarrow \{0,1\} \tag{129}$$

such that for every $G = (V, E)$ and $k \in \mathbb{N}$, $VC(G, k) = 1$ if and only if there exists a vertex cover $S \subset V$ such that $|S| \leq k$.

Theorem 10.4 ($3SAT \leq_p VC$)

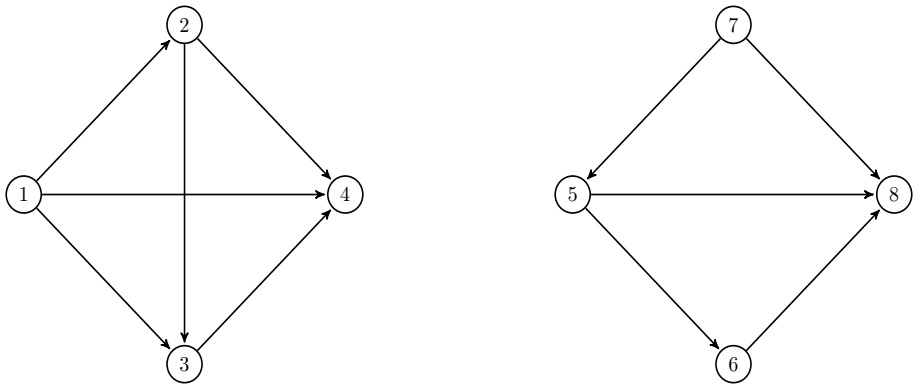
$$3SAT \leq_p VC \tag{130}$$

Definition 10.5 (Clique)

A **clique** is a subset of vertices of an undirected graph such that every two distinct vertices in the graph are adjacent, i.e. connected by an edge.
The **maximum clique problem** corresponds to the function

$$CLIQUE : \{0,1\}^* \longrightarrow \{0,1\} \tag{131}$$

such that for a graph G and a number k, $CLIQUE(G, k) = 1$ iff there is a subset S of k vertices such that for *every* distinct $u, v \in S$, the edge u, v is in G. For example, in the graph below, the left subset of 4 vertices is indeed a clique, while the right subset of 4 is not since the edge connecting 6 to 7 is not present.



Theorem 10.5 (Equivalence of CLIQUE and ISET)

$$CLIQUE \leq_p ISET \text{ and } ISET \leq_p CLIQUE \tag{132}$$

Definition 10.6 (Dominating Set)

A **dominating set** in a graph $G = (V, E)$ is a subset $S \subset V$ of vertices such that for every $u \in V \setminus S$ is a neighbor in G

10.3.1 Anatomy of a Reduction

A reduction from problem F to a problem G is an algorithm that maps an input x for F to an input $R(x)$ for G. To show that the reduction is correct we need to show the properties of:

1. *efficiency*: algorithm R runs in polynomial time
2. *completeness*: if $F(x) = 1$, then $G(R(x)) = 1$
3. *soundness*: if $F(R(x)) = 1$, then $G(x) = 1$

Therefore, proving that problem G is a reduction of problem F is equivalent to showing the three properties above.

We finally reduce the 3SAT problem to the longest path problem.

Theorem 10.6 (Hardness of Longest Path)

$$3SAT \leq_p LONGPATH \tag{133}$$

That is, an efficient algorithm for the *longest path* problem would imply a polynomial-time algorithm for 3SAT. Therefore, we have shown that 3SAT is no harder than Quadratic Equations, Independent Set, Maximum Cut, and Longest Path.

11 NP, NP Completeness, and Cook-Levin Theorem

All of the problems that we have talked about are *search problems*, where the goal is to decide, given an instance x, whether there exists a solution y that satisfies some condition that can verified in polynomial time. For example, in 3SAT, the instance is a formula and the solution is an assignment to the variable; in Max-Cut the instance is a graph and the solution is a cut in the graph; and so on and so forth. It turns out that every such search problem can be reduced to 3SAT.

11.1 The Class NP

Intuitively, the class **NP** corresponds to the class of problems where it is *easy to verify* a solution (i.e. verification can be done by a polynomial-time algorithm). For example, finding a satisfying assignment to a 2SAT or 3SAT formula is such a problem, since if we are given an assignment to the variables of a 2SAT or 3SAT formula then we can efficiently verify that it satisfies all constraints.

That is, a Boolean function F is in **NP** if F has the form that on input string x, $F(x) = 1$ if and only if there exists a "solution" string w such that the pair (x, w) satisfies some polynomial-time checkable condition.

Definition 11.1 (NP - Nondeterministic Polynomial Time)

We say that $F : \{0,1\}^* \longrightarrow \{0,1\}$ is in **NP** if there exists some integer $a > 0$ and $V : \{0,1\}^* \longrightarrow \{0,1\}$ such that $V \in \mathbf{P}$ and for every $x \in \{0,1\}^n$,

$$F(x) = 1 \iff \text{there exists } w \in \{0,1\}^{n^a} s.t.\ V(xw) = 1 \tag{134}$$

That is, for F to be in **NP**, there needs to exist some polynomial time computable verification function V such that if $F(x) = 1$, then there must exist w (of length polynomial in $|x|$) such that $V(xw) = 1$, and if $F(x) = 0$ then for *every* such w, $V(xw) = 0$. Since the existence of this string w certifies that $F(x) = 1$, w is often called the *certificate, witness*, or *proof* that $F(x) = 1$.

Some problems that are NP are:

1. $3SAT \in \mathbf{NP}$ since for every l-variable formula φ, $3SAT(\varphi) = 1$ if and only there exists a satisfying assignment $x \in \{0,1\}^l$ such that $\varphi(x) = 1$, and we can check this condition in polynomial time.
2. $QUADEQ \in \mathbf{NP}$ since for every l-variable instance of quadratic equations E, $QUADEQ(E) = 1$ if and only if there exists an assignment $x \in \{0,1\}^l$ that satisfies E. We can check the condition that x satisfies E in polynomial time by enumerating over all the equations in E, and for each such equation e, plug in the values of x and verify that e is satisfied.
3. $ISET \in \mathbf{NP}$ since for every graph G and integer k, $ISET(G,k) = 1$ if and only if there exists a set S of k vertices that contains no pair of neighbors in G. We can check the condition that S is an independent set of size $\geq k$ in polynomial time by first checking that $|S| \geq k$ and then enumerating over all edges $\{u,v\}$ in G, and for each such edge verify that either $u \neq S$ or $v \neq S$.
4. $LONGPATH \in \mathbf{NP}$ since for every graph G and integer k, $LONGPATH(G,k) = 1$ if and only if there exists a simple path P in G that is of length at least k. We can check the condition that P is a simple path of length k in polynomial time by checking that it has the form $(v_0, v_1, ..., v_k)$ where each v_i is a vertex in G, no v_i is repeated, and for every $i \in [k]$, the edge $\{v_i, v_{i+1}\}$ is present in the graph.
5. $MAXCUT \in \mathbf{NP}$ since for every graph G and integer k, $MAXCUT(G,k) = 1$ if and only if there exists a cut $(S, \overline{S})$ in G that cuts at least k edges. We can check that condition that $(S, \overline{S})$ is a cut of value at least k in polynomial time by checking that S is a subset of G's vertices and enumerating over all the edges $\{u,v\}$ of G, counting those edges such that $u \in S$ and $v \notin S$ or vice versa.

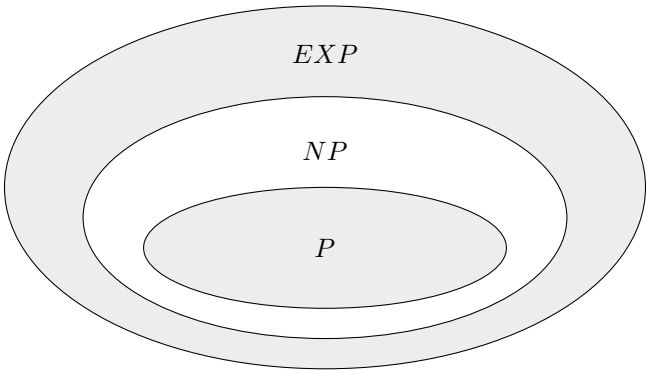
Theorem 11.1 ($\mathbf{P} \subseteq \mathbf{NP}$)

Verifying is no harder than solving:

$$\mathbf{P} \subseteq \mathbf{NP} \tag{135}$$

Furthermore,

$$\mathbf{P} \subseteq \mathbf{NP} \subseteq \mathbf{EXP} \tag{136}$$



Proof. Suppose that $F \in \mathbf{P}$. Define the following function V:

$$V(x0^n) = \begin{cases} 1 & \text{iff } n = |x|, F(x) = 1 \\ 0 & \text{else} \end{cases} \tag{137}$$

Since $F \in \mathbf{P}$, we can clearly compute V in polynomial time as well. Let $x \in \{0,1\}^n$ be some string. If $F(x) = 1$ then $V(x0^n) = 1$. On the other hand, if $F(x) = 0$ then for every $w \in \{0,1\}^n$, $V(xw) = 0$. Therefore, setting $a = 1$ (i.e. $w \in \{0,1\}^{n^1}$), we see that V satisfies the NP condition.

11.2 NP Hard and NP Complete Problems

There are countless examples of problems for which we do not know if their best algorithm is polynomial or exponential, but we can show that they are in **NP**; that is, we don't know if they are easy to *solve*, but we do know that it is easy to *verify* a given solution. There are many other functions that we would like to compute that are easily shown to be in **NP**. In fact, it we can solve 3SAT then we can solve all of them!

Theorem 11.2 (Cook-Levin Theorem)

For every $F \in \mathbf{NP}$,

$$F \leq_p 3SAT \tag{138}$$

This immediately implies that $QUADEQ, LONGPATH$, and $MAXCUT$ (and really, *every* $F \in \mathbf{NP}$) all reduce to $3SAT$, meaning that all these problems are equivalent! All of these problems are the "hardest in **NP**" since an efficient algorithm for any one of them would imply an efficient algorithm for *all* the problems in **NP**.

Definition 11.2 (NP-Hard)

Let $G : \{0,1\}^* \longrightarrow \{0,1\}$. We say that G is **NP hard** if for every $F \in \mathbf{NP}$, $F \leq_p G$. We say that G is **NP complete** if G is **NP** hard and $G \in \mathbf{NP}$.

Therefore, despite their differences, 3SAT, quadratic equations, longest path, independent set, maximum cut, and thousands of other problems are all **NP** complete. Again, this means that *if a single **NP** complete problem has a polynomial-time algorithm, then there is such a polynomial-time algorithm for every decision problem that corresponds to the existence of an efficiently verifiable solution (i.e. is NP), which would imply that* $\boldsymbol{P = NP}$.

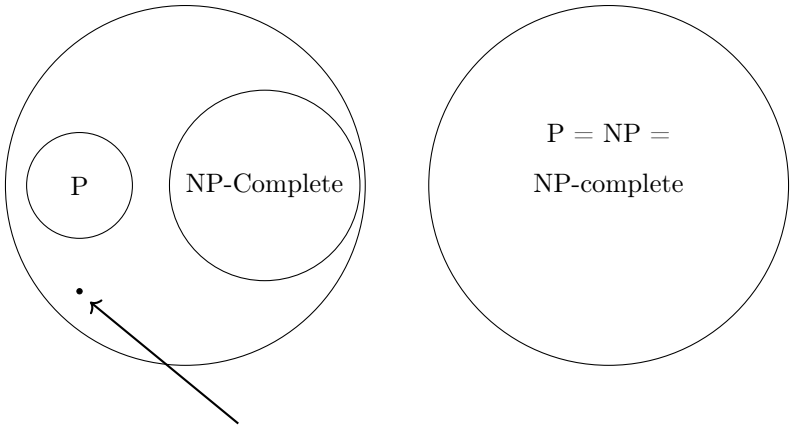
11.3 P = NP?

However, a polynomial-time algorithm for even a single one of the **NP** complete problems has even been found, proving support that $\mathbf{P} \neq \mathbf{NP}$

One of the mysteries of computation is that people have observed a certain empirical "zero-one law" or "dichotomy" in the computational complexity of natural problems, in the sense that many natural problems are either in **P** (often in $TIME(O(n))$ or $TIME(O(n^2))$), or they are **NP** hard. This is related to the fact that for most natural problems, the best known algorithm is either exponential or polynomial, rather than any strange function in between.

However, it is believed that there exist problems in **NP** that are neither in **P** nor are **NP** complete, and in fact a result known as **Lander's Theorem** shows that if $\mathbf{P} \neq \mathbf{NP}$, then this is indeed the case. Therefore, we are left with two cases:

1. If $\mathbf{P} \neq \mathbf{NP}$, meaning that **P** is a strict subset of **NP** and by Lander's theorem, **NP** complete problems do not cover all of $\mathbf{NP} \setminus \mathbf{P}$. (left)
2. If $\mathbf{P} = \mathbf{NP}$, meaning that $\mathbf{P} = \mathbf{NP} = \mathbf{NP}$ complete. (right)



problem that is neither P nor NP complete (by Lander's Theorem)

11.4 NANDSAT, 3NAND Problems

Definition 11.3 (The NANDSAT Problem)

The function $NANDSAT : \{0,1\}^* \longrightarrow \{0,1\}$ is defined as follows:

1. The input to $NANDSAT$ is a string Q representing a NAND-CIRC program (or equivalently, a circuit with $NAND$ gates)
2. The output of $NANDSAT$ on input Q is 1 if and only if there exists a string $w \in \{0,1\}^n$ (where n is the number of inputs to Q) such that $Q(w) = 1$.

Definition 11.4 (The 3NAND Problem)

The $3NAND$ problem is defined as follows:

1. The input is a logical formula Ψ on a set of variables $z_0, ..., z_{r-1}$ which is an AND of constraints of the form $z_i = NAND(z_j, z_k)$.
2. The output is 1 is and only if there is an input $z \in \{0,1\}^r$ that satisfies all of the constraints.

Example 11.1 (A 3NAND Formula)

The following is a $3NAND$ formula with 5 variables and 3 constraints:

$$\Psi = \big(z_3 = NAND(z_0, z_2)\big) \wedge \big(z_1 = NAND(z_0, z_2)\big) \wedge \big(z_4 = NAND(z_3, z_1)\big) \tag{139}$$

In this case $3NAND(\Psi) = 1$, since the assignment $z = 01010$ satisfies it. Given a $3NAND$ formula Ψ of r variables and an assignment $z \in \{0,1\}^r$, we can check in polynomial time whether $\Psi(z) = 1$, and hence $3NAND \in \mathbf{NP}$.

Theorem 11.3 (NP-Completeness of NANDSAT and 3NAND)

$NANDSAT$ and $3NAND$ are **NP**-complete.

12 Intractability

12.1 Uncomputable Functions

Even though NAND-CIRC programs can compute every finite function $f : \{0,1\}^n \longrightarrow \{0,1\}$, NAND-TM programs can *not* compute every function $F : \{0,1\}^* \longrightarrow \{0,1\}$. That is, there exists such a function that is *uncomputable*!

Definition 12.1 (The Halting Problem)

Let $HALT : \{0,1\}^* \longrightarrow \{0,1\}$ be the function such that for every string $M \in \{0,1\}^*$, $HALT(M,x) = 1$ if Turing machine M halts on the input x and $HALT(M,x) = 0$ otherwise.

Theorem 12.1 (HALT Is Not Computable)

The $HALT$ function is not computable. This leads to many other functions also being uncomputable.

It is surprising that such a simple program is actually uncomputable. That is, there is no *general procedure* that would determine for an *arbitrary* program P whether it halts or not.

12.2 Impossibility of General Software Verification

Definition 12.2 (Semantic Property)

Let there be a program P that computes a function. A **semantic property** or **semantic specification** of a program means properties of the *function* that the program computes, as opposed to the properties that depend on the particular syntax/code used by the program.

Example 12.1 (A Semantic Property of a Program)

A semantic property of a program P is the property that whenever P is given an input string with an even number of 1's, it outputs 0. Another example is the property that P will always halt whenever the input ends with a 1.
In contrast the property that a C program contains a comment before every function declaration is not a semantic property, since it depends on the actual source code as opposed to the input/output relation.

Example 12.2 (Two Programs Computing the Same Function)

Consider the following two C programs:

```
int First(int n) {
    if (n<0) return 0;
    return 2*n;
}

int Second(int n) {
    int i = 0;
    int j = 0
    if (n<0) return 0;
```

```
10      while (j<n) {
11          i = i + 2;
12          j= j + 1;
13      }
14      return i;
15  }
```

`First` and `Second` are two distinct C programs, but they compute the same function. Therefore, a *semantic property* would either be true for both programs or false for both, since it depends on the function the programs compute. One example of a semantic property is: *The program P computes a function f mapping integers to integers satisfying that $f(n) \geq n$ for every input n.*
A property is *not semantic* if it depends on the source code rather than the input/output behavior. An example of this would be: *The program contains the variable* `k` or *the program uses the* `while` *operation.*

Definition 12.3 (Semantic properties)

A pair of Turing machines M and M' are **functionally equivalent** if for every $x \in \{0,1\}^*$, $M(x) = M'(x)$ (including when the function outputs $\perp$).
A function $F : \{0,1\}^* \longrightarrow \{0,1\}$ is **semantic** if for every pair of strings M, M' that represent functionally equivalent Turing machines, $F(M) = F(M')$. Note that we assume that every string represents *some* Turing machine.

We now present a theorem concerning the Halting problem (the problem of determining whether a Turing machine will halt or not on any arbitrary input). The Halting problem also turns out to be a linchpin of uncomputability.

Theorem 12.2 (Rice's Theorem)

Let $F : \{0,1\}^* \longrightarrow \{0,1\}$. If F is semantic and nontrivial, then it is uncomputable.

Corollary 12.3 (COMPUTES-PARITY Is Uncomputable)

The following function is uncomputable:

$$COMPUTES-PARITY(P) = \begin{cases} 1 & P \text{ computes the parity function} \\ 0 & \text{else} \end{cases} \tag{140}$$

Therefore, we can see that the set $\mathbf{R}$ of computable Boolean functions is a proper subset of the set of all functions mapping $\{0,1\}^* \longrightarrow \{0,1\}$.

13 Probabilistic Computation

It turns out that randomness can actually be a resource for computation, enabling us to achieve tasks much more efficiently than previously known. This advantage comes from the idea that calculating the statistics of a system could be done much faster by running several randomized simulations rather than explicit calculations, and these types of randomized algorithms are known as *Monte Carlo algorithms*.

13.1 Finding Approximately Good Maximum Cuts

Recall the maximum cut problem of finding, given a graph $G = (V, E)$, the cut that maximizes the number of edges. This problem is **NP**-hard, which means that we do not know of any efficient algorithm that can solve it, but randomization enables a simple algorithm that can cut at least half of the edges.

Theorem 13.1 (Approximating Max Cut)

There is an efficient probabilistic algorithm that on input an n-vertex m-edge graph G, outputs a cut $(S, \overline{S})$ that cuts at least $m/2$ of the edges of G in expectation.

Proof. We simply choose a *random cut*: we choose a subset S of vertices by choosing every vertex v to be a member of S with probability $1/2$ independently. More specifically, upon input of a graph $G = (V, E)$ with vertices $(v_0, ..., v_{n-1})$, we do

1. Pick x uniformly at random in $\{0, 1\}^n$
2. Let $S \subseteq V$ be the set $\{v_i \mid x_i = 1, i \in [n]\}$ that includes all vertices corresponding to coordinates of x where $x_i = 1$.
3. Output the cut $(S, \overline{S})$.

We claim that the expected number of edges cut by the algorithm is $m/2$. Indeed, for every edge $e \in E$, let X_e be the random variable such that $X_e(x) = 1$ if the edge is cut by x, and let $X_e(x) = 0$ otherwise. It is not hard to see that the probability of $X_e(x) = 1$ is $\frac{1}{2}$ (when exactly one of the vertices are in S), and hence

$$\mathbb{E}(X_e) = 1/2 \tag{141}$$

Summing this over all edges and by linearity of expectation, we get

$$\mathbb{E}(X) = \sum_{e \in E} \mathbb{E}(X_e) = m \cdot \frac{1}{2} = \frac{m}{2} \tag{142}$$

In fact, for *every graph* G, the algorithm is guaranteed to cut half of the edges of the input graph in expectation.

13.1.1 Amplifying the success of randomized algorithms

But note that expectation does not imply concentration. Luckily, we can *amplify* the probability of success by repeating the process several times and outputting the best cut we find. We assume that the probability that the algorithm above succeeds in cutting at least $m/2$ edges is not *too* tiny.

Lemma 13.2 (A Random Cut Cuts Half the Edges)

The probability that a random cut in an m edge graph cuts at least $m/2$ edges is at least $\frac{1}{2m}$.

Proof. This is quite trivial when looking at specific cases. For example, take the case when $m = 1000$ edges. In this case, one can shot that we will cut at least 500 edges with probability at least 0.001 (and so in particular larger then $\frac{1}{2m} = \frac{1}{2000}$). Specifically, if we assume otherwise, then this means that with probability more than 0.999 the algorithm cuts 499 or fewer edges. But since we can never cut more

than the total of 1000 edges, given this assumption, the highest value of the expected number of edges cut is if we cut exactly 499 edges with probability 0.999 and cut 1000 edges with probability 0.001. But this leads to the expectation being

$$0.999 \cdot 499 + 0.001 \cdot 1000 < 500 \tag{143}$$

which contradicts the fact that the expectation to be at least 500 in the previous theorem. Generalizing this to m edges, we find that the expected number of edges cut is

$$pm + (1-p)\Big(\frac{m}{2} - \frac{1}{2}\Big) \leq pm + \frac{m}{2} - \frac{1}{2} \tag{144}$$

But since $p < \frac{1}{2m} \implies pm < 0.5$, the right hand side is smaller than $m/2$, contradicting the fact that the expected number of edges cut is at least $m/2$.

13.1.2 Success Amplification

To increase the chances of success, we simply need to repeat our program many times, with fresh randomness each time, and output the best cut we get in one of these repetitions. It turns our that if we repeat this experiment $2000m$ times, then by using the inequality

$$\Big(1 - \frac{1}{k}\Big)^k \leq \frac{1}{e} \leq \frac{1}{2} \tag{145}$$

we can show that the probability that we will never cut at least $m/2$ edges is at most

$$\Big(1 - \frac{1}{2m}\Big)^{2000m} \leq 2^{-1000} \tag{146}$$

This can be generalized in the following lemma.

Lemma 13.3 (Randomized Max-Cut Approximation)

There is an algorithm that on input graph $G = (V, E)$ and a number k, runs in polynomial time in $|V|$ and k and outputs a cut $(S\overline{S})$ such that

$$\mathbb{P}\Big(\text{number of edges cut by } (S, \overline{S}) \geq \frac{|E|}{2}\Big) \geq 1 - 2^{-k} \tag{147}$$

Proof. Just repeat the previous algorithm $200km$ times and compute the probability of failure.

13.1.3 Two-sided Amplification

The analysis above relied on the fact that the maximum has *one sided error*; that is, if we get a cut of size at least $m/2$ then we know we have succeeded. This is common for randomized algorithms, but it is not the only case. In particular, consider the task of computing some Boolean function $F : \{0,1\}^* \longrightarrow \{0,1\}$. A randomized algorithm A for computing F, given input x, might toss coins and succeed in outputting $F(x)$ with probability, say 0.9. We say that A has *two sided errors* if there is a positive probability that $A(x)$ outputs 1 when $F(x) = 0$ and positive probability that $A(x)$ outputs 0 when $F(x) = 1$. So, we cannot simply repeat it k times and output 1 if a single one of those repetitions resulted in 1, nor can we output 0 if a single one of the repetitions resulted in 0. But we can output the *majority value* of these repetitions: the probability that the fraction of the repetitions where A will output $F(x)$ will be at least, say 0.89, will be exceptionally close to 1 and in such cases we will output the correct answer.

Theorem 13.4 (Amplification of One-Sided Error)

If $F : \{0,1\}^* \longrightarrow \{0,1\}$ is a function such that there is a polynomial-time algorithm A satisfying

$$\mathbb{P}\big(A(x) = F(x)\big) \geq 0.51 \tag{148}$$

for every $x \in \{0,1\}^*$, then there is a polynomial time algorithm B satisfying

$$\mathbb{P}\big(B(x) = F(x)\big) \geq 1 - 2^{-|x|} \tag{149}$$

for every $x \in \{0,1\}^*$.

13.1.4 Solving SAT through Randomization

The 3SAT problem is **NP** hard, and so it is unlikely that it has a polynomial (or even subexponential) time algorithm. But this does not mean that we can't do at least somewhat better than the trivial 2^n algorithm for n-variable 3SAT. The best known worst-case algorithms are randomized and are at their base the following simple algorithm. In this algorithm, called *WalkSAT*, the input is an n variable 3CNF formula φ, the parameters are any numbers $T, S \in \mathbb{N}$, and the operation is:

1. Repeat the following T steps:
   (a) Choose a random assignment $x \in \{0,1\}^n$ and repeat the following for S steps:
      i. If x satisfies φ, then output x.
      ii. Otherwise, choose a random clause $(l_i \vee l_j \vee l_k)$ that x does not satisfy, choose a random literal in $l_i \vee l_j \vee l_k$ and modify x to satisfy this literal.
2. If all the $T \cdot S$ repetitions above did not result in a satisfying assignment, then output `Unsatisfiable`.

Note that we are only going though at most $S \cdot T$ configurations of $x \in \{0,1\}^n$, and the running time of this algorithm is $S \cdot T \cdot \mathit{poly}(n)$. The fact that this algorithm is efficient is taken care of, so now the key question is how small we can make S and T so that the probability that WalkSAT outputs `Unsatisfiable` on a satisfiable formula φ is small. It is known that we can do with

$$ST = \tilde{O}\big((4/3)^n\big) = \tilde{O}(1.\overline{3}^n) \tag{150}$$

However, we will prove a weaker bound in the following theorem (which is still much better than the 2^n bound).

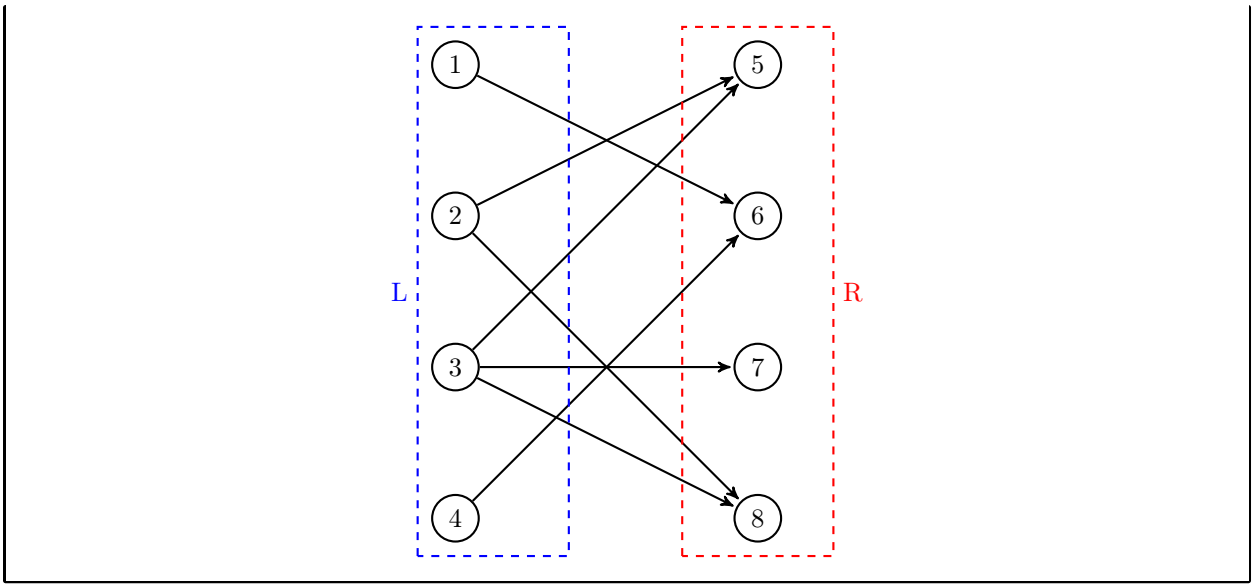
Theorem 13.5 (WalkSAT simple analysis)

If we set $T = 100\sqrt{3}^n$ and $S = n/2$, then the probability we output `Unsatisfiable` for a satisfiable φ is at most $\frac{1}{2}$.

13.1.5 Bipartite Matching

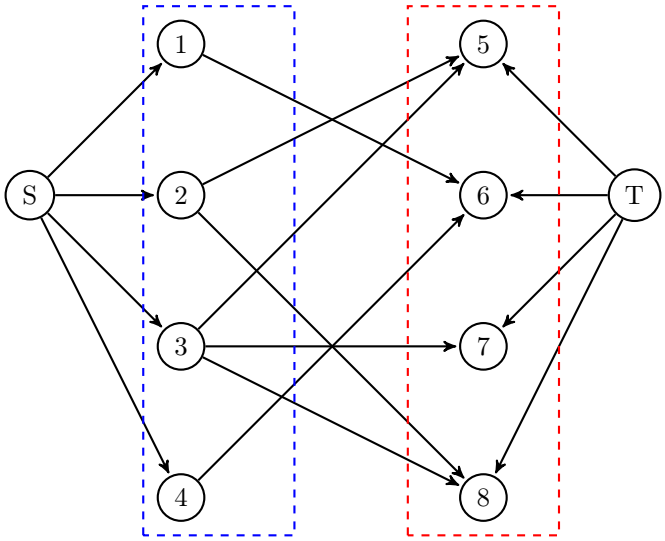
Definition 13.1 (Bipartite Graph)

A **bipartite graph** $G = (L \cup R, E)$ has $2n$ vertices partitioned into n-sized sets L and R, where all edges have one endpoint in L and the other in R.



A *matching problem* is a type of problem where we match nodes to each other with edges. One variant of it is called the *bipartite perfect matching*. The goal is to determine whether there is a *perfect matching*, a subset $M \subseteq E$ of n disjoint edges that connects every vertex L to a unique vertex in R.

It turns out that by reducing this problem of finding a matching in G to finding a maximum flow (or equivalently, a minimum s, t cut) in a related graph G' (below), we can solve it in polynomial time.



However, there is a different probabilistic algorithm to do this. Let G's vertices be labeled as $L = \{l_0, ..., l_{n-1}\}$ and $R = \{r_0, ..., r_{n-1}\}$. A matching M corresponds to a *permutation* $\pi \in S_n$ where for ever $i \in [n]$, we define $\pi(i)$ to be the unique j such that M contains the edge $\{l_i, r_j\}$. Define an $n \times n$ matrix $A = A(G)$ where $A_{i,j} = 1$ if and only if $\{l_i, r_j\}$ is present and $A_{i,j} = 0$ otherwise. The correspondence between matchings and permutations implies the following claim.

Lemma 13.6 (Matching polynomial)

Define $P = P(G)$ to be the polynomial mapping $\mathbb{R}^{n^2}$ to $\mathbb{R}$ where

$$P(x_{0,0}, ..., x_{n-1,n-1} = \sum_{\pi \in S_n} \left(\prod_{i=0}^{n-1} sign(\pi) A_{i,\pi(i)} \right) \prod_{i=0}^{n-1} x_{i,\pi(i)} \tag{151}$$

In fact, given the matrix A representing the graph, the polynomial above is the determinant of the matrix $A(x)$, which is obtained by replaying $A_{i,j}$ with $A_{i,j}x_{i,j}$. Then G has a perfect matching if and only if P is not identically zero (i.e. if there exists some assignment $x = (x_{i,j})_{i,j \in [n]} \in \mathbb{R}^{n^2}$ such that $P(x) \neq 0$.

This reduces testing perfect matching to testing whether a given polynomial $P(\cdot)$ is identically 0 or not. The kernel of most multivariate nonzero polynomials form a strictly lower dimensional space than the total space, so in order to do this, we just choose a "random" input x and check if $P(x) \neq 0$. However, to transform this into an actual algorithm, we can't work in the real numbers with our finite computational power. We use the following.

Theorem 13.7 (Schwartz-Zippel Lemma)

For every integer q and polynomial $P : \mathbb{R}^n \longrightarrow \mathbb{R}$ with integer coefficients, if P has degree at most d and is not identically zero, then it has at most dq^{n-1} roots in the set

$$[q]^n = \left\{ (x_0, ..., x_{n-1}) \mid x_i \in \{0, 1, ..., q-1\} \right\} \tag{152}$$

Therefore, upon an input of a bipartite graph G on $2n$ vertices $\{l_0, ..., l_{n-1}, r_0, ..., r_{n-1}\}$, the *Perfect-Matching algorithm* can be divided into these steps:

1. For every $i, j \in [n]$, choose $x_{i,j}$ independently at random from $[2n] = \{0, ..., 2n-1\}$.
2. Compute the determinant of the matrix $A(x)$ whose i, jth entry equals $x_{i,j}$ if the edge $\{l_i, r_j\}$ is present and 0 otherwise.
3. Output `no perfect matching` if determinant is 0, and output `perfect matching` otherwise.

13.2 Modeling Randomized Computation

While we have described randomized algorithms in an informal way, we haven't addressed two questions:

1. How do we actually efficiently obtain random strings in the physical world?
2. What is the mathematical model for randomized computations, and is it more powerful than deterministic computation?

The first question is important, but we will assume that there are various physical sources of random or unpredictable data, such as a user's mouse movements, network latency, thermal noise, and radioactive decay. For example, many Intel chips come with a random number generator built in. We will focus on the second question.

Modeling randomized computation is actually quite easy. We can add the operation

```
foo = RAND()
```
(153)

in addition to things like the NAND operator to any programming language such as NAND-TM, NAND-RAM, NAND-CIRC, etc., where `foo` is assigned to a random bit in $\{0, 1\}$ independently every time it is called. These are called RNAND-TM, RNAND-RAM, and RNAND-CIRC, respectively.

Similarly, we can easily define randomized Turing machines as Turing machines in which the transition function δ gets an extra input (in addition to the current state and symbol read from the tape) a bit b that in each step is chosen at random in $\{0,1\}$. Of course the function can ignore this bit (and have the same output regardless of whether $b = 0$ or $b = 1$) and hence randomized Turing machines generalize deterministic Turing machines.

We can use the `RAND()` operation to define the notion of a function being computed by a randomized $T(n)$ time algorithm for every nice time bound $T : \mathbb{N} \longrightarrow \mathbb{N}$, but we will only define the class of functions that are computable by randomized algorithms running in *polynomial time*.

Definition 13.2 (The class BPP)

Let $F : \{0,1\}^* \longrightarrow \{0,1\}$. We say that $F \in \mathbf{BPP}$ if there exist constants $a, b \in \mathbb{N}$ and a RNAND-TM program P such that for every $x \in \{0,1\}^*$, on input x, the program P halts within at most $a|x|^b$ steps and

$$\mathbb{P}\big(P(x) = F(x)\big) \geq \frac{2}{3} \tag{154}$$

where this probabilty is taken over the result of the RAND operations of P. Note that this probability is taken only over the random choices in the execution of P and *not* over the choice of the input x. That is, **BPP** is still a *worst case* complexity class, in the sense that if F is in **BPP** then there is a polynomial-time randomized algorithm that computes F with probability at least 2/3 on *every possible* (and not just random) input.

We will use the name *polynomial time randomized algorithm* to denote a computation that can be modeled by a polynomial-time RNAND-TM program, RNAND-RAM program, or a randomized Turing machine.

Alternatively, we can think of a randomized algorithm A as a *deterministic algorithm* A' that takes two inputs x and r where the input r is chosen at random from $\{0,1\}^m$ for some $m \in \mathbb{N}$. The equivalence to the previous definition is shown in the following theorem:

Definition 13.3 (Alternative characterization of BPP)

Let $F : \{0,1\}^* \longrightarrow \{0,1\}$. Then $F \in \mathbf{BPP}$ if and only if there exists $a, b \in \mathbb{N}$ and $G : \{0,1\}^* \longrightarrow \{0,1\}$ such that G is in $\mathbf{P}$ and for every $x \in \{0,1\}^*$,

$$\mathbb{P}\big(G(xr) = F(x)\big) \geq \frac{2}{3} \tag{155}$$

where r is chosen at random from $\{0,1\}^{a|x|^b}$. As such, if A is a randomized algorithm that on inputs of length n makes at most m coin tosses, we will often use the notation $A(x;r)$ (where $x \in \{0,1\}^n$ and $r \in \{0,1\}^m$ to refer to the result of executing x when the coin tosses of A correspond to the coordinates of r. This second input r is sometimes called a **random tape**.

The relationship between **BPP** and **NP** is not known, but we do know the following.

Theorem 13.8 (Sipser-Gacs Theorem)

If $\mathbf{P} = \mathbf{NP}$ then $\mathbf{BPP} = \mathbf{P}$.

13.2.1 Success Amplification of two-sided error algorithms

The number 2/3 may seem arbitrary, but it can be amplified to our liking.

Theorem 13.9 (Amplification)

Let $F : \{0,1\}^* \longrightarrow \{0,1\}$ be a Boolean function such that there is a polynomial $p : \mathbb{N} \longrightarrow \mathbb{N}$ and a polynomial-time randomized algorithm A satisfying that for every $x \in \{0,1\}^n$,

$$\mathbb{P}\big(A(x) = F(x)\big) \geq \frac{1}{2} + \frac{1}{p(n)} \tag{156}$$

Then for every polynomial $q : \mathbb{N} \longrightarrow \mathbb{N}$, there is a polynomial-time randomized algorithm B satisfying for every $x \in \{0,1\}^n$,

$$\mathbb{P}\big(B(x) = F(x)\big) \geq 1 - 2^{-q(n)} \tag{157}$$

13.2.2 BPP and NP Completeness

The theory of **NP** completeness still applies to probabilistic algorithms.

Theorem 13.10 (NP-Hardness inside BPP Collapses the Hierarchy)

Suppose that F is **NP** hard and $F \in \mathbf{BPP}$. Then

$$\mathbf{NP} \subseteq \mathbf{BPP} \tag{158}$$

That is, if there was a randomized polynomial time algorithm for any **NP** complete problem such as 3SAT, ISET, etc., then there would be such an algorithm for *every* problem in **NP**.

13.3 The Power of Randomization

To find out whether randomization can add power to computation (does **BPP**=**P**?), we prove a few statements about the relationship of **BPP** with other complexity classes.

Theorem 13.11 (Simulating randomized algorithms in exponential time)

$$\mathbf{BPP} \subseteq \mathbf{EXP} \tag{159}$$

Proof. We can just enumerate over all the (exponentially many) choices for the random coins.

Furthermore,

$$\mathbf{P} \subseteq \mathbf{BPP} \subseteq \mathbf{EXP} \tag{160}$$